گزارش فنی — آموزشی

تحلیل کتابخانه‌های سیستم حضور و غیاب نوین پرداز

کالبدشکافی کامل ۵۱ فایل بستهٔ NovinDll's.zip با دکامپایلر ILSpy — آموزش لایه به لایه به همراه کدهای واقعی

NOVIN PARDAZ TIME & ATTENDANCE — .NET LIBRARY ANALYSIS REPORT

در این گزارش، ۱۳ کتابخانهٔ اختصاصی نرم‌افزار جامع حضور و غیاب و حقوق و دستمزد «نوین پرداز» (پروژهٔ NPACC) را لایه‌به‌لایه بررسی می‌کنیم: از لایهٔ دسترسی داده و بانک اطلاعاتی SQL Server تا درایورهای شش برند دستگاه اثر انگشت، رابط کاربری WinForms، چاپ Stimulsoft، مودم صوتی و سرویس پیامک — همیشه با کد واقعیِ دکامپایل‌شده و نگاه معلم‌وار.

.NET Framework 4.8 | C# / WinForms | SQL Server | SignalR | Stimulsoft

۵۱ فایل • ۳۴٫۸ MB | ۶ برند دستگاه

نسخهٔ ۱٫۰ — آبان ۱۴۰۵

سند آموزشی عمومی — تهیه‌شده بر پایهٔ تحلیل ایستا (Static Analysis)

ILSpy Decompiler

NovinDll's.zip / file.kiwi

نقشهٔ راه

TOC

در این گزارش چه می‌خوانیم؟

سیزده فصل پیشِ رو به ترتیبِ «از کلان به جزئی» چیده شده‌اند. اگر تازه‌کار هستید، ترتیب را به هم نزنید؛ اگر حرفه‌ای هستید، می‌توانید مستقیم به فصل کتابخانهٔ موردنظر بپرید. شمارهٔ صفحهٔ آغازین هر فصل در گوشهٔ کارت‌ها آمده است.

01 — مقدمه؛ بسته‌ای که بررسی کردیم — دسته‌بندی ۵۱ فایل و روش تحلیل — صفحهٔ ۱

02 — معماری کلان سیستم — دیاگرام لایه‌ها و سفر یک تردد — صفحهٔ ۳

03 — TimeAttendanceDAL — قلب سیستم: بانک، DAL، رمزنگاری، ابر — صفحهٔ ۵

04 — TimeAttendanceDevices — درایورهای شش برند دستگاه اثر انگشت — صفحهٔ ۱۴

05 — TimeAttendanceUI — ۱۵۰ فرم، ورود، مانیتورینگ زنده — صفحهٔ ۲۲

06 — TimeAttendancePrint — چاپ و گزارش‌سازی با Stimulsoft — صفحهٔ ۲۵

07 — ValidatorAttribute — قواعد اعتبارسنجی موجودیت‌ها — صفحهٔ ۲۷

08 — Tele.dll — مودم صوتی و تردد تلفنی با AT — صفحهٔ ۲۹

09 — WebSmsStandard — کلاینت REST سرویس پیامک — صفحهٔ ۳۱

10 — ZK و ZDC2911 و zkemkeeper — کتابخانه‌های بومی و ابزارها — صفحهٔ ۳۴

11 — کتابخانه‌های جانبی — اعلان شیشه‌ای و انیمیشن — صفحهٔ ۳۶

12 — کتابخانه‌های System.* — زیرساخت مایکروسافت (۲۶ فایل) — صفحهٔ ۳۷

13 — جمع‌بندی و امنیت — درس‌های معماری و یافته‌های امنیتی — صفحهٔ ۳۸

فصل یکم

CH-01

مقدمه؛ بسته‌ای که بررسی کردیم

Introduction

در این گزارش آموزشی، محتوای فایل فشرده **NovinDll's.zip** (که از طریق سرویس اشتراک‌گذاری file.kiwi دریافت شد) را موجوده به موشکافی کدهای درونی آن بررسی می‌کنیم. این بسته شامل **۵۱ فایل** با حجم کل حدود **۳۴٫۸ مگابایت** است و در واقع پوشهٔ خروجی (Bin) یک نرم‌افزار تجاری ایرانی به نام **«نرم‌افزار جامع حضور و غیاب و حقوق و دستمزد نوین پرداز»** می‌باشد؛ نام پروژهٔ این نرم‌افزار در دل کدها **NPACC** است و توسعه‌دهندهٔ آن شرکت «نوین پرداز» (مقرر در مشهد، پیش‌شمارهٔ تلفن ۰۵۱) است. هدف ما این است که مثل یک کلاس درس، لایه به لایه و کتابخانه به کتابخانه، بفهمیم این نرم‌افزار چگونه ساخته شده، هر قطعه چه وظیفه‌ای دارد و کدهای واقعی داخل آن چه می‌گویند.

همهٔ کدهایی که در این گزارش می‌بینید، از داخل خود فایل‌های DLL با ابزار **ILSpy** (دیکامپایلر متن‌باز دات‌نت) استخراج شده‌اند؛ یعنی هیچ کد فرضی یا ساختگی در کار نیست و هر قطعه کد را می‌توانید با مسیر فایل منبعش مقایسه کنید. بسته‌بندی فنی نرم‌افزار هم از خود فایل‌ها قابل مشاهده است: **.NET Framework نسخهٔ 4.8**، زبان **C#**، رابط کاربری **WinForms** با کامپوننت‌های DevComponents، و پایگاه‌دادهٔ **Microsoft SQL Server**.

> 💡 **کد دکامپایل‌شده یعنی چه؟**
>
> وقتی یک برنامهٔ C# ساخته می‌شود، کد شما به زبان میانی IL (Intermediate Language) ترجمه و داخل فایل DLL قرار می‌گیرد. دیکامپایلرها مثل ILSpy همین زبان میانی را دوباره به C# برمی‌گردانند. به همین دلیل کدهای این گزارش گاهی شکل «عجیبی» دارند: پرانتزهای اضافه مثل `((DbCommand)(object)cmd)`، متغیرهای بی‌استفاده مثل `_ = 2;` یا کلمهٔ `num` به‌جای نام‌های اصلی متغیرها — این‌ها artefact (اثر جانبی) دکامپایل هستند، نه سبک برنامه‌نویسی نویسندهٔ اصلی. برای خوانایی، کامنت‌های `//IL_...` را حذف کرده‌ایم اما منطق کد دست‌نخورده است.

آنچه داخل بسته پیدا کردیم

بسته را باز کردیم و فایل‌ها را بر اساس نقش‌شان در چهار گروه دسته‌بندی کردیم. نکتهٔ جالب اینکه فایل اجرایی اصلی `TimeAttendance.exe` در این بسته **حاضر نیست** و فقط فایل‌های `TimeAttendance.exe.config` و `TimeAttendance.pdb` از آن باقی مانده‌اند؛ یعنی فرستندهٔ بسته فقط کتابخانه‌ها را جمع کرده است. فایل‌های `.pdb` هم اطلاعات عیب‌یابی (خط و نام فایل‌های سورس اصلی) هستند و همین فایل‌ها بودند که مسیر پروژهٔ اصلی توسعه‌دهنده را لو دادند: `D:\Projects\NPACC\WebService_Team\TimeAttendance\...`

جدول ۱-۱: دسته‌بندی ۵۱ فایل موجود در بسته

گروه	تعداد	فایل‌ها	توضیح

کتابخانه‌های اختصاصی اصلی	۷	`TimeAttendanceDAL`
		`TimeAttendanceUI`
		`TimeAttendanceDevices`
		`TimeAttendancePrint`
		`TimeAttendance.ValidatorAttribute`
		`WebSmsStandard`
		`Tele`
			قلب نرم‌افزار؛ لایهٔ دسترسی داده، رابط کاربری، درایور دستگاه‌ها، چاپ، اعتبارسنجی، پیامک و تلفن

ابزارها و درایورهای کمکی	۴	`zkemkeeper.dll`، `ZDC2911.dll`، `ZK.dll`، `zkemsdk.dll`	درایور/SDK سازندگان دستگاه‌های اثر انگشت (ZKTeco و Realand)
کتابخانه‌های متن‌باز جانبی	۲	`Tulpep.NotificationWindow`، `WinFormAnimation`	نمایش اعلان‌های شیشه‌ای و انیمیشن فرم‌ها (از NuGet)
کتابخانه‌های مایکروسافتی	۲۶ + ۱۲	`System.*.dll` (۲۶ فایل) و فایل‌های `.pdb` / `.config` (۱۲ فایل)	زیرساخت استاندارد .NET Framework که در فصل ۱۲ مرور می‌کنیم

۵۱	۱۳	۳۴٫۸	+۴۵۰
فایل در بسته	کتابخانهٔ اختصاصی و کمکی	مگابایت حجم کل	کلاس دکامپایل‌شده

۶
برند دستگاه پشتیبانی‌شده

چطور این گزارش را بخوانیم؟

گزارش طوری چیده شده که مثل یک دورهٔ آموزشی از بالا به پایین جلو برود: ابتدا در فصل ۲ تصویر کلان معماری را می‌بینیم، سپس در فصل‌های ۳ تا ۱۱ وارد جزئیات هر کتابخانه می‌شویم — همیشه ابتدا «این کتابخانه چه مشکلی را حل می‌کند؟» بعد «چطور کار می‌کند؟» و سپس «کد واقعی‌اش را ببینیم». در فصل ۱۲ کتابخانه‌های استاندارد مایکروسافت را جدولی مرور می‌کنیم و در فصل ۱۳ درس‌های معماری و یافته‌های امنیتی را جمع‌بندی می‌کنیم. جعبه‌های آبی «نکتهٔ معلم» مفاهیم پایه را توضیح می‌دهند و جعبه‌های قرمز «هشدار امنیتی» نقاط ضعفی را نشان می‌دهند که از بابت آموزش امنیت، دیدنشان برای شما ارزشمند است.

فصل دوم

CH-02

معماری کلان سیستم؛ نقشهٔ راه فنی

System Architecture

قبل از اینکه وارد هر کتابخانه شویم، باید نقشهٔ کلان را ببینیم. این نرم‌افزار یک **معماری چندلایه (Multi-Layer / n-Tier)** کلاسیک دارد؛ یعنی مسئولیت‌ها بین لایه‌های مجزا تقسیم شده‌اند تا هر لایه فقط کار خودش را انجام دهد و تغییر یک لایه، بقیه را خراب نکند. اگر هرگز با این مفهوم آشنا نبوده‌اید، نکتهٔ معلم زیر را بخوانید.

💡 معماری چندلایه یعنی چه؟

معماری چندلایه را با یک رستوران مقایسه کنید: **سالن و منو** (رابط کاربر) سفارش مشتری را می‌گیرد، **گارسن** (لایهٔ میانی/اعتبارسنجی) سفارش را بررسی و به آشپزخانه می‌رساند، **آشپزخانه** (لایهٔ منطق/دسترسی داده) غذا را آماده می‌کند و **انبار** (پایگاه‌داده) مواد اولیه را نگه می‌دارد. اگر مدیر رستوران منوی سالن را عوض کند، نیازی نیست آشپزخانه را بازسازی کند — هر لایه فقط از طریق «سفارش‌های استاندارد» (در دنیای نرم‌افزار: متدها و اینترفیس‌ها) با هم حرف می‌زنند.

نقشهٔ لایه‌ها

دیاگرام زیر را با دقت ببینید؛ در تمام گزارش به همین نقشه برگشت می‌کنیم. جهت خواندن از بالا (کاربر) به پایین (سخت‌افزار) است.

لایهٔ ۱ — رابط کاربر (Presentation)

فرم‌های ویندوزی، ورود کاربران، گزارش‌ها، مانیتورینگ زندهٔ ترددها

TimeAttendanceUI.dll (۱۵ مگابایت، ~۱۵۰ فرم) | TimeAttendance.exe (در این بسته غایب است)

▼

لایهٔ ۲ — خدمات کنار رابط کاربر

اعتبارسنجی ورودی‌ها، چاپ گزارش‌ها و اعلان‌های شیشه‌ای

TimeAttendance.ValidatorAttribute | TimeAttendancePrint (موتور Stimulsoft)

Tulpep + WinFormAnimation

▼



جدول ۲-۱: نقش هر کتابخانه در معماری

لایه	کتابخانه	با چه چیزی حرف می‌زند	فصل مرتبط
رابط کاربر	`TimeAttendanceUI`	DAL (هرگز مستقیم با SQL حرف نمی‌زند)	فصل ۵
اعتبارسنجی	`ValidatorAttribute`	DAL + فرم نمایش خطا	فصل ۷
چاپ	`TimeAttendancePrint`	Stimulsoft + DAL	فصل ۶
دسترسی داده	`TimeAttendanceDAL`	SQL Server با ADO.NET + سرور ابری با SignalR	فصل ۳
درایور دستگاه	`TimeAttendanceDevices`	SDK هر برند (COM / P/Invoke / TCP)	فصل ۴
پیامک	`WebSmsStandard`	سرویس REST در webservice.novinp.ir	فصل ۹
تلفن	`Tele`	پورت سریال و دستورات AT مودم	فصل ۸

سفر یک «تردد» از انگشت تا فیش حقوقی

بهترین روش فهم یک سیستم حضور و غیاب، دنبال کردن مسیر یک رخداد واقعی است. فرض کنید کارمند «رضا» ساعت ۸ صبح انگشتش را روی دستگاه می‌گذارد؛ اتفاقی که در طول چند ثانیه تا چند هفته می‌افتد این است:

۱ — ثبت در دستگاه
دستگاه اثر انگشت، شمارهٔ کاربر + زمان + نوع ورودی/خروجی را در حافظهٔ خود ذخیره می‌کند

۲ — دریافت توسط درایور
درایور برند مربوطه در TimeAttendanceDevices رکوردها را از دستگاه می‌خواند

۳ — یکسان‌سازی
کلاس RecordAllDevice رکورد هر برند را به موجودیت استاندارد DeviceLogOnPerson تبدیل می‌کند

۴ — ذخیره در بانک
DAL آن را با روال ذخیره‌شدهٔ sp_LogonPersonel_Insert در جدول LogonPersonel ثبت می‌کند

۵ — تطبیق هویت
کد سخت‌افزاری (HardwareCode) کاربر با جدول اشخاص تطبیق داده می‌شود و PersonId مشخص می‌شود

۶

محاسبهٔ کارکرد

TaradodFullDet ورود/خروج‌ها را جفت می‌کند و کارکرد، اضافه‌کار و کسری‌کار روزانه را می‌سازد

۷

فیش حقوقی

کلاس Fish بر اساس قرارداد و فرمول‌ها، فیش حقوقی ماهانه تولید می‌کند

۸

خروجی

چاپ فیش (Print)، ارسال پیامک به مدیر (WebSms) یا اعلان زندهٔ روی مانیتور (UI)

💡 **واژه‌نامهٔ دامنهٔ حضور و غیاب**

در سیستم‌های حضور و غیاب، به هر رخداد ورود یا خروج یک «تردد» (Traffic) گفته می‌شود و جدول/کلاس آن اینجا LogOnPersonel نام دارد. عبارت «سخت‌افزار کد» (HardwareCode) هم یعنی شمارهٔ کاربری که روی دستگاه اثر انگشت برای هر شخص تعریف شده است. چون ممکن است یک شخص در چند دستگاه با شماره‌های مختلف ثبت شده باشد، سیستم جدولی جداگانه (HardwareCodeCus) برای نگاشت «شمارهٔ دستگاه + کد روی دستگاه ← شخص» دارد.

کلید جادویی: contextGuid

در تمام کدهای این نرم‌افزار مدام با پارامتری به نام contextGuid برخواهید خورد — مثلاً Person.GetAllAsync(contextGuid, ...). این یک شناسهٔ یکتا (GUID) است که مشخص می‌کند عملیات روی «کدام یک از زمینه‌های داده» انجام شود. نرم‌افزار به این ترتیب می‌تواند هم‌زمان چند شرکت/بانک مختلف را مدیریت کند، هر چند در عمل معمولاً یک بانک فعال است. این کلید در تمام لایه‌ها از UI تا DAL ترد می‌شود و حکم «شناسهٔ جلسهٔ فعال» را دارد؛ به همین دلیل تقریباً همهٔ کلاس‌ها اینترفیس IHasContext را پیاده‌سازی کرده‌اند.

🔎 **نام‌گذاری جالب بانک‌ها**

نام فایل‌های بانک اطلاعاتی این سیستم جالب است: هنگام نصب، DAL دنبال اولین نام خالی از Attendance0 تا Attendance99 می‌گردد و بانک را با آن نام می‌سازد! این منطق را در فصل ۳ با کد واقعی خواهیم دید.

فصل سوم

CH-03

کتابخانهٔ TimeAttendanceDAL؛ قلب تپندهٔ سیستم

TimeAttendanceDAL.dll — Data Access Layer

این کتابخانه با حجم **۶٫۳ مگابایت**، **۱۹۳ فایل سورس** و بیش از ۳۰ فضای‌نام (Namespace)، بزرگ‌ترین و مهم‌ترین قطعهٔ نرم‌افزار است. هر عملی که ذخیره‌سازی، خواندن، محاسبه یا ارتباط با دنیای خارج لازم دارد، یا مستقیماً اینجا انجام می‌شود یا از اینجا فراخوانی می‌شود: از ثبت ترددها و مدیریت پرسنل و قراردادها تا فیش حقوقی، بیمه، مالیات، پیامک‌های خودکار و همگام‌سازی با سرور ابری شرکت. در کدهای این کتابخانه حتی مشخصات سخت‌افزاری «قفل نرم‌افزار» (لایسنس USB) هم بررسی می‌شود.

جدول ۳-۱: نقشهٔ فضاهای‌نام اصلی DAL (FLR = Filter/List/Report)

فضای‌نام	مسئولیت	کلاس‌های شاخص
TimeAttendanceDAL	مدیریت رشتهٔ اتصال	ClsConnection
...Database	ساخت و بازسازی بانک	CLSDatabase، RunScript، Bazsazi
...Dal.FLRPerson	مدیریت پرسنل	Person + ۹ کلاس جدول‌های کمکی
...Dal.Taradod	ترددها، مرخصی‌ها و محاسبات	LogOnPersonel، TaradodFullDet
...Dal.FLRGharardad	قراردادهای کاری	GharardadNew، FormulaClass

فضای‌نام	مسئولیت	کلاس‌های شاخص
`...Dal.Salary`	حقوق و دستمزد	Fish (۸۵ کیلوبایت کد!)، TaxTable
`...Dal.FLRShift`	شیفت‌ها	Shifts، ShiftDet
`...Dal.FLRSanad`	اسناد حسابداری	AttSanad، ATT_SanadDet
`...Dal.Device`	دستگاه‌ها و کدهای سخت‌افزاری	DeviceInfo، HardwareCodeCus
`...Dal.User`	کاربران و سطوح دسترسی	UserAttendance، UserPermission
`...Settings`	تنظیمات متمرکز (JSON در بانک)	TimeAttendanceSettings، Payamak
`...Utility`	ابزارهای عمومی	Utility، FMessageBox، AttLogger
`...Mapper`	تبدیل موجودیت‌ها به مدل ابری	PersonMapper و ۱۶ مپر دیگر

۳-۱) اتصال به پایگاه‌داده؛ سبک کلاسیک ADO.NET

DAL با پایگاه‌داده **SQL Server** و از طریق کلاس‌های ADO.NET کار می‌کند: `SqlConnection` برای اتصال، `SqlCommand` برای فرمان و `SqlDataReader` برای خواندن نتیجه. نکتهٔ معماری این است که **هیچ اتصال بلندمدتی نگه داشته نمی‌شود**؛ هر متد، اتصال خودش را می‌سازد، کارش را انجام می‌دهد و در بلوک `finally` آن را آزاد می‌کند. مدیریت اتصال‌های باز در قالب «استخر اتصال» (Connection Pooling) خود ADO.NET به‌صورت خودکار انجام می‌شود.

رشتهٔ اتصال در کلاس `ClsConnection` نگهداری می‌شود. متد جالب آن `CheckConnectionString` است که یک اتصال آزمایشی می‌سازد تا از درستی رشتهٔ اتصال مطمئن شود:

TimeAttendanceDAL/ClsConnection.cs — اعتبارسنجی رشتهٔ اتصال

```
public static bool CheckConnectionString(string constr)
{
    try
    {
        SqlConnectionStringBuilder val = new SqlConnectionStringBuilder(constr)
        {
            AsynchronousProcessing = true
        };
        SqlConnection val2 = new SqlConnection();
        ((DbConnection)val2).ConnectionString =
            ((DbConnectionStringBuilder)(object)val).ConnectionString;
        AttLogger.LoggerUi.Information("checkConnectionStrindg() ... ");
        ((DbConnection)val2).Open();
        AttLogger.LoggerUi.Information("checkConnectionStrindg() is ok");
        ((DbConnection)val2).Close();
        connectionString = constr;
        return true;
    }
    catch (Exception ex)
    {
        WebErrorLog.ErrorLogInstance.StartLog(ex.Message, "connstr=" + constr,
            "CheckConnectionString", "...\ClsConnection.cs", 45);
        return false;
    }
}
```

توجه کنید که استثناها (Exception) هرگز باعث کرش برنامه نمی‌شوند؛ همه‌جا با try/catch گرفته شده و در WebErrorLog ثبت می‌شوند — رویه‌ای که در سراسر این نرم‌افزار تکرار می‌شود.

اما چگونه خود بانک ساخته می‌شود؟ متد `CreateDbIfNotExist` مثل کارگر نصب عمل می‌کند: از نام `Attendance0` شروع می‌کند و تا `Attendance99` جلو می‌رود تا اولین نام آزاد را بیابد:

TimeAttendanceDAL.Database/CLSDatabase.cs — ساخت خودکار بانک با نام Attendance0 تا 99

```
public static async Task<ReturnedSaveFuncInfo> CreateDbIfNotExist(string contextGuid)
{
    ReturnedSaveFuncInfo ret = new ReturnedSaveFuncInfo();
    if (Extensions.GetSqlConnectionString(contextGuid) != "")
    {
        return ret;
    }
    string Database = "Attendance";
    int i = 0;
    try
    {
        for (; i < 100; i++)
        {
            string connectionString = (await BaseDataBaseAction.Instance
                .CreateDataBaseAsync(BackupSettingsRepo.GetInstance(contextGuid),
                                     Database + i)).ConnectionString;
            if (!string.IsNullOrEmpty(connectionString) &&
                ClsConnection.CheckConnectionString(connectionString))
            {
                ClsCache cache = ClsCache.GetCache(contextGuid);
                if (cache != null)
                {
                    cache.Dispose();
                }
                CacheContext context = new CacheContext($"CreateDataBaseGuid{Guid.NewGuid()}");
                await EntityCacheConfig.InitAsync(context.ContextGuid, connectionString,
                    SystemInfoClass.SystemInfo(), VersionManagment.Version,
                    (EnConnectionType)1);
                ClsConnection.ContextGuid = context.ContextGuid;
                return ret;
            }
        }
    }
    catch (Exception ex)
    {
        ret.AddReturnedValue((ReturnedState)2, "ایجاد بانک اطلاعاتی با خطا مواجه شد", ...);
        WebErrorLog.ErrorLogInstance.StartLog(ex, "", "CreateDbIfNotExist", ...);
    }
    ret.AddReturnedValue((ReturnedState)2, "بانک اطلاعاتی ایجاد نشد", ...);
    return ret;
}
```

پیام‌های خطا فارسی هستند چون مستقیم به کاربر نهایی نمایش داده می‌شوند. ساختار ReturnedSaveFuncInfo حامل وضعیت (موفق/خطا/هشدار) + پیام فارسی است.

پس از ساخت بانک خالی، کلاس `RunScript` کل شِمای پایگاه‌داده را از فایل‌های ریسورس داخل خود DLL اجرا می‌کند و به ترتیب این‌ها را می‌سازد: توابع SQL، جدول‌ها، تریگرها، ویوها، روال‌های ذخیره‌شده و حتی **جدول تقویم (CreateTaghvim)** که داده‌های تقویم شمسی — از جمله تعطیلات رسمی ایران — را پر می‌کند. جالب است بدانید این متد حتی نسخهٔ SQL Server را هم چک می‌کند: اگر سرور قدیمی‌تر از 2012 باشد، خطای نبودِ تابع `FORMAT()` را می‌بیند و پیام «Sql server version must be higher than 2012» نمایش می‌دهد. کلاس `Bazsazi` («بازسازی») هم همین زنجیره را برای ارتقا/مهاجرت نسخه‌های قدیمی اجرا می‌کند و در پایان حتی یک پینگ به آدرس 8.8.8.8 می‌زند تا اتصال اینترنت را بسنجد!

> 💡 **روال ذخیره‌شده (SP) چیست؟**
>
> **روال ذخیره‌شده (Stored Procedure)** یک برنامهٔ کوچک داخل خود SQL Server است که با دستور CREATE PROCEDURE ساخته می‌شود. مزیتش سه‌تاست: ۱) سرعت بالاتر چون از قبل کامپایل شده، ۲) کاهش رفت‌وبرگشت شبکه، ۳) امنیت بیشتر در برابر تزریق SQL اگر پارامترها را درست پاس بدهیم. این نرم‌افزار تقریباً ۹۵٪ کارهایش را با SP انجام می‌دهد و قاعدهٔ نام‌گذاری‌اش کاملاً یکنواخت است: `sp_نام‌جدول_عملیات` مثلاً `sp_Person_Insert`، `sp_Person_Update`، `sp_Person_SelectAll`.

۳-۲) کالبدشکافی یک کلاس DAL؛ الگوی Active Record

تقریباً همهٔ کلاس‌های این کتابخانه یک الگوی واحد دارند که در ادبیات مهندسی نرم‌افزار به آن **Active Record** می‌گویند: «هر شیء = یک ردیف از جدول». یعنی یک نمونه از کلاس Person یک کارمند واقعی است؛ اگر `SaveAsync()` را صدا بزنید و `Id` صفر باشد **درج** می‌کند و اگر Id مقدار داشته باشد **به‌روزرسانی**. متدهای استاتیک هم نقش «فهرست‌ها و جستجوها» را دارند. بیایید این منطق را در کد واقعی ببینیم:

TimeAttendanceDAL.Dal.FLRPerson/Person.cs — قلب متد SaveAsync

```
await ((DbConnection)(object)cn).OpenAsync(token);
if (((PersonBase)this).Id <= 0)
{
    // جدید است → روال درج را صدا بزن
    ((DbCommand)(object)cmd).CommandText = "[dbo].[sp_Person_Insert]";
    object obj = await ((DbCommand)(object)cmd).ExecuteScalarAsync(token);
    if (obj != null)
    {
        ((PersonBase)this).Id = Convert.ToInt32(obj.ToString());
    }
    await LogUsers.RegisterNewLogAsync(EntityCacheConfig.CacheInstance.ContextGuid,
        "[افزودن اطلاعات یک شخص جدید] : " + ((PersonBase)this).FullName +
        " [با کد تفصیلی ] : " + ((PersonBase)this).CodeTafsil,
        "Personnel.PersonnelInsert", this, ((PersonBase)this).Id.ToString());
}
else
{
    // موجود است → روال ویرایش را صدا بزن
    ((DbCommand)(object)cmd).CommandText = "[dbo].[sp_Person_Update]";
    cmd.Parameters.AddWithValue("ID", (object)((PersonBase)this).Id);
    await ((DbCommand)(object)cmd).ExecuteNonQueryAsync(token);
    await LogUsers.RegisterNewLogAsync(EntityCacheConfig.CacheInstance.ContextGuid,
        "[ویرایش اطلاعات پرسنلی] : " + ((PersonBase)this).FullName, ...);
}
((DbConnection)(object)cn).Close();
Reset();
return true;
```

بعد از هر تغییر، یک رکورد «حسابرسی» (Audit) هم در جدول LogUsers ثبت می‌شود؛ یعنی چه‌کسی، چه زمانی، چه چیزی را تغییر داده — قابلیتی که برای نرم‌افزارهای منابع انسانی تقریباً الزامی است.

نکتهٔ ظریف این کلاس، وجود یک **کش ۶۰ ثانیه‌ای** روی فهرست کل اشخاص است. چون تقریباً همهٔ فرم‌ها به لیست پرسنل نیاز دارند، دوباره‌خوانی آن از SQL Server در هر فرم، پرهزینه است؛ پس نتیجهٔ آخرین کوئری در یک متغیر استاتیک نگه داشته می‌شود و فقط اگر ۶۰ ثانیه از آخرین بارگذاری گذشته باشد، دوباره از بانک خوانده می‌شود:

Person.cs — کش استاتیک ۶۰ ثانیه‌ای جلوی sp_Person_SelectAll

```
public static async Task<List<Person>> GetAllAsync(string contextGuid,
    List<int> listPersonId, bool personStatus, CancellationToken token)
{
    try
```

```
    {
        if (LastUpdateTime.AddSeconds(60.0) < DateTime.Now)
        {
            LastUpdateTime = DateTime.Now;
            List<Person> ret = new List<Person>();
            SqlConnection cn = new SqlConnection(
                Extensions.GetSqlConnectionString(contextGuid));
            SqlCommand cmd = new SqlCommand("[sp_Person_SelectAll]", cn);
            ((DbCommand)(object)cmd).CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("personIDs", (object)"");
            await ((DbConnection)(object)cn).OpenAsync(token);
            SqlDataReader dr = await cmd.ExecuteReaderAsync(token);
            while (await ((DbDataReader)(object)dr).ReadAsync(token))
            {
                token.ThrowIfCancellationRequested();
                ret.Add(LoadData(contextGuid, dr));
            }
            ((DbDataReader)(object)dr).Close();
            Persons = ret;      // کش استاتیک به‌روز شد
        }
        List<Person> list = Persons;
        if (personStatus)
        {
            list = Persons.Where((Person p) =>
                p != null && ((PersonBase)p).St).ToList();
        }
        return list;
    }
    ...
}
```

> **💡 LoadData چه می‌کند؟**
>
> متد `LoadData(SqlDataReader)` که در کد بالا می‌بینید «هیدراتور» (Hydrator) نام دارد: یک ردیف خوانده‌شده از بانک را می‌گیرد و فیلدهایش را یکی‌یکی در پراپرتی‌های شیء Person می‌ریزد — مثل خالی‌کردن آب در قالب. الگوی `try/finally` با `Dispose` هم تضمین می‌کند منابع (اتصال، فرمان، خواننده) حتی هنگام خطا آزاد شوند.

۳-۳) موجودیت تردد؛ LogOnPersonel

کلاس `LogOnPersonel` (موجودیت جدول LogonPersonel) پرترافیک‌ترین کلاس سیستم است چون هر کارت‌زدنی یک ردیف از آن است. ذخیره‌سازی آن دو ویژگی مهم دارد: نخست از یک **سمافور (SemaphoreSlim)** استفاده می‌شود تا در لحظهٔ ذخیره‌های موازی، دو تردد هم‌زمان تداخل نکنند؛ دوم پس از ذخیره، شبکهٔ آخرین ترددهای پایش زنده در UI هم به‌روزرسانی می‌شود:

LogOnPersonel.cs — ذخیرهٔ یک تردد با روال ذخیره‌شده (قسمتی از متد SaveAsync)

```
cmd.Parameters.AddWithValue("DateTimeLogM", (object)DateTimeLogM);
cmd.Parameters.AddWithValue("IDTraffic", (object)TrafficId);
cmd.Parameters.AddWithValue("IsVirtual", (object)(short)(int)IsVirtual);
cmd.Parameters.AddWithValue("TrafficType", (object)(short)(int)TypeTaradod);
cmd.Parameters.AddWithValue("TrafficName",
    (Utilities.GetDisplay<TypeTaradodEnumType>(TypeTaradod) + " / " +
     Utilities.GetDisplay<TypeTaradodEnumType>(TypeTaradodVacationOrMission)));
await ((DbConnection)(object)cn).OpenAsync(token);
if (Id <= 0)
{
    cmd.Parameters.AddWithValue("PersonId", (object)PersonId);
    cmd.Parameters.AddWithValue("DeviceId", (object)DeviceId_);
    cmd.Parameters.AddWithValue("DeviceSerial", (object)DeviceSerial_);
    ((DbCommand)(object)cmd).CommandText = "sp_LogonPersonel_Insert";
    SqlDataReader dr = await cmd.ExecuteReaderAsync(token);
```

```
    if (await ((DbDataReader)(object)dr).ReadAsync(token) &&
        ((DbDataReader)(object)dr)["id"] != DBNull.Value)
    {
        Id = (int)((DbDataReader)(object)dr)["id"];
    }
}
else
{
    ((DbCommand)(object)cmd).CommandText = "sp_LogonPersonel_Update";
    cmd.Parameters.AddWithValue("Id", (object)Id);
    await ((DbCommand)(object)cmd).ExecuteNonQueryAsync(token);
}
((DbConnection)(object)cn).Close();
TimeAttendanceDAL.Utility.Utility.ReloadDgridLastTaradodsAsync(ContextGuid, token);
return Id;
```

خط آخر یعنی: بعد از هر ذخیره، گرید «آخرین ترددها» روی فرم اصلی دوباره پر شود — اتصال بی‌واسطهٔ DAL به UI از طریق یک متد استاتیک.

اعتبارسنجی این کلاس هم کاملاً «ایرانی» است؛ متد `Validatetion` پیام‌های فارسی تولید می‌کند: ترددی با زمانِ آینده پذیرفته نمی‌شود، ترددِ متعلق به دستگاه ناشناس رد می‌شود، کد شخص باید شناخته‌شده باشد و ترددهای قدیمی‌تر از ۹۰ روز هم باید به‌صورت دستی تأیید شوند.

۳-۴) کوئری‌ها؛ پارامتری در برابر الحاقی

همان‌طور که گفتیم اکثر کوئری‌ها از طریق SP و **پارامتری** اجرا می‌شوند (روش امن). اما جاهایی کدنویس برای کوئری‌های کوچک، رشته‌ها را مستقیم به هم چسبانده است. یک نمونهٔ سالم و یک نمونهٔ خطرناک:

Person.cs — کوئری پارامتری (روش درست)

```
// نمونهٔ سالم: پارامتری — مقدار خارج از متن کوئری منتقل می‌شود
cmd.Parameters.AddWithValue("@CodeMoein", (object)codeM);
cmd.Parameters.AddWithValue("@CodeTafsil", (object)codeT);
((DbCommand)(object)cmd).CommandText =
    "Select Count(*) from Person Where CodeMoein=@CodeMoein " +
    "and CodeTafsil=@CodeTafsil and ID<>@ID and St='True'";
```

Shifts.cs — کوئری الحاقی (روش خطرناک)

```
// نمونهٔ ناامن: الحاق رشته — مستعد تزریق SQL
SqlCommand cmd = new SqlCommand(
    "select Id from [shifts] where [St]='true' and Code='" + code + "'", cn);
```

> ⚠️ **تزریق SQL (SQL Injection)**
>
> در کلاس Shifts متد `UniqeCodeAsync` مقدار پارامتر `code` را مستقیم داخل متن SQL می‌چسباند. اگر مقدار ورودی از کاربر تأمین شود، مهاجم می‌تواند با تزریق عبارتی مثل `1' OR '1'='1` منطق کوئری را دور بزند. نمونه‌های مشابه در `Person.LoadEnrollDataAsync` (الحاق Id عددی — کم‌خطر)، `FishDet` و `ShiftProgrammableDet` هم دیده می‌شود. درس آموزشی: **همیشه از پارامتر استفاده کنید، حتی برای اعداد.**

۳-۵) رمزنگاری؛ AES با یک کلیدِ داخل کد

سطوح دسترسی و رمز عبور کاربران نرم‌افزار، قبل از ذخیره در بانک با الگوریتم **AES/Rijndael** رمز می‌شوند. پیاده‌سازی فنی آن کاملاً حرفه‌ای است: استخراج کلید با PBKDF2 (تابع Rfc2898DeriveBytes با ۱۰۰۰ تکرار)، مقدار نم (Salt) و بردار آغازین (IV) تصادفی و الحاق آن‌ها به ابتدای خروجی:

TimeAttendanceDAL.Classes/Encryption.cs — رمزنگاری AES-256 با Salt تصادفی

```
public static string Encrypt(string plainText, string passPhrase)
{
    byte[] array = Generate256BitsOfRandomEntropy();   // Salt تصادفی
    byte[] array2 = Generate256BitsOfRandomEntropy();  // IV تصادفی
    byte[] bytes = Encoding.UTF8.GetBytes(plainText);
    using Rfc2898DeriveBytes rfc2898DeriveBytes =
        new Rfc2898DeriveBytes(passPhrase, array, 1000);
    byte[] bytes2 = rfc2898DeriveBytes.GetBytes(32);   // کلید ۲۵۶ بیتی
    using RijndaelManaged rijndaelManaged = new RijndaelManaged();
    rijndaelManaged.BlockSize = 256;
    rijndaelManaged.Mode = CipherMode.CBC;
    rijndaelManaged.Padding = PaddingMode.PKCS7;
    using ICryptoTransform transform =
        rijndaelManaged.CreateEncryptor(bytes2, array2);
    using MemoryStream memoryStream = new MemoryStream();
    using CryptoStream cryptoStream =
        new CryptoStream(memoryStream, transform, CryptoStreamMode.Write);
    cryptoStream.Write(bytes, 0, bytes.Length);
    cryptoStream.FlushFinalBlock();
    byte[] inArray = array.Concat(array2).ToArray()
        .Concat(memoryStream.ToArray()).ToArray();
    return Convert.ToBase64String(inArray);
}
```

⚠️ **کلید هاردکدشده در کد!**

مشکل کجاست؟ در **عبور رمز**! کلاس UserPermission این متد را با یک عبارت رمز ثابت و هاردکد صدا می‌زند: `private readonly string _passPhrase = "NovinP****************";` — یعنی هرکسی که به همین DLL یا سورس دکامپایل‌شدهٔ آن دسترسی داشته باشد (دقیقاً مثل ما!) می‌تواند محتوای رمزشدهٔ سطوح دسترسی را باز کند. رمزنگاری قوی با کلید ضعیف، مثل قفل فولادیِ روی درِ پارچه‌ای است. راه درست: نگهداری کلید در ماژول امن (HSM/TPM) یا حداقل استفاده از DPAPI ویندوز.

۳-۶) پل ارتباطی با ابر؛ SignalR

یکی از جذاب‌ترین بخش‌های DAL، لایهٔ همگام‌سازی ابری است. نرم‌افزار با هاب **SignalR** سرور نوین پرداز (آدرس پیش‌فرض: `https://novinrank.ir/hubCore`) وارد می‌شود و شناسه‌های خود را معرفی می‌کند: نام کاربری NPACC، شمارهٔ سریال قفل سخت‌افزاری LK، شناسهٔ پردازنده و GUID بانک. بعد از این «ورود»، ترددهایی که در پنل تحت‌وب ثبت می‌شوند، به‌صورت زنده به مشتری محلی هدایت می‌شوند:

پیکربندی ورود به هاب ابری — TimeAttendanceDAL.Settings/EntityCacheConfig.cs

```
List<ClientConfig> list = new List<ClientConfig>
{
    new ClientConfig
    {
        UserName = usernameSr,
        Password = passwordSr,
        Source = (ENSource)5,
        DbGuid = dbGuid,
        Version = version,
        CompressedMode = (ENCompressedMode)1,
        LkSerial = lkSerial,
        CpuSerial = systemInfo,
        Url = url
    }
};
_cacheInstance.Context.LoginExtention.OnLoginResponse -= Instance_OnLoginResponse;
_cacheInstance.Context.LoginExtention.OnLoginResponse += Instance_OnLoginResponse;
...
ReturnedSaveFuncInfoWithValue<LoginResponseEventArgs> val =
    await cacheInstance.ConnectAsync(connectionType, list, token2, true);
```

دریافت دسته‌ای ترددها از سرور ابری — EntityCacheConfig.cs

```
private static async Task WebAttendanceLogOnPerson_OnSync(object sender,
    ConnectionEventArgs<WebAttendanceLogOnPerson> e)
{
    try
    {
        Utility.MessageBoxShowByNotif(showPopUp: false, (NotificationEnum)9,
            "3 ,"" ,"شروع عملیات سینک");
        ReturnedSaveFuncInfoWithValue<(int, List<string>)> val =
            await SaveTaradod(stringBuilder,
                e.Connection.Context.Guid, e.ConnectionData,
                e.Connection.cancellationToken);
        if (val.value.Item1 > 0)
        {
            Utility.MessageBoxShowByNotif(showPopUp: false, (NotificationEnum)7,
                string.Format("اتمام عملیات سینک\r\n [تعداد ترددهای دریافت شده]: {0}",
                    val.value.Item1), "[اتمام عملیات سینک]", 10);
        }
        ...
    }
}
```

همچنین یک سرویس ویندوزی به نام NpCommunicationDeviceService (با هاب BioMetricServerCore) روی پورت 5001 نصب می‌شود که رلهٔ ارتباط دستگاه‌های زیستی با این هاب است.

🔎 سینک دورهای ۳۰ دقیقه‌ای

DAL هر ۳۰ دقیقه یکبار اشخاصِ تغییریافتهٔ محلی را به سرور ابری می‌فرستد و تغییرات ثبت‌شده در پنل وب را برمی‌گرداند (دو جهتِ یک سینک). برای هر کارمند هم یک لینک ورود به «سلف‌سرویس» ساخته می‌شود: `https://nphozor.ir/Login/...` با یک هش بی‌اس۶۴ از نام‌کاربری + رمز + GUID شخص.

۳-۷) الگوهای طراحیِ قابل مشاهده در DAL

جدول ۳-۲: الگوهای طراحی شناسایی‌شده در DAL

الگو	کجا دیده می‌شود	توضیح کوتاه
Active Record	همهٔ کلاس‌های Dal.*	هر شیء یک ردیف جدول با متدهای Save/Delete

الگو	کجا دیده می‌شود	توضیح کوتاه
Singleton (برای هر context)	EnrollManage، TimeAttendanceSettings	یک نمونهٔ یکتا در ConcurrentDictionary با کلید contextGuid
Repository	DeviceLogOnPersonRepository	تنها ریپازیتوری صریح؛ ذخیرهٔ تردد دریافتی از دستگاه
Mapper / DTO	فضای‌نام Mapper با ۱۷ کلاس	تبدیل موجودیت محلی به مدل ابری WebAtt*
Observer (رویدادها)	OnSaveTaradod، OnStatusErorTaradod	اعلان تغییرات به UI بدون وابستگی مستقیم
Retry (تلاش مجدد)	SaveSmsRefrencedIdsAsync	تا ۲۰ بار با فاصلهٔ ۵ ثانیه، به‌صورت بازگشتی!
Producer/Consumer	BufferBlock در Utility	صف اعلان‌ها با TPL Dataflow
کش با انقضا	Person.Persons + LastUpdateTime	کش ۶۰ ثانیه‌ای فهرست اشخاص
JSON Blob	JsonGharardadDet، JsonFishDet، تنظیمات	ذخیرهٔ جزئیات اسکیماپذیر به‌صورت JSON در یک ستون

LogOnPersonel.cs — الگوی Retry (بازگشتی (۲۰ بار × ۵ ثانیه)

```
public async Task SaveSmsRefrencedIdsAsync(string smsRefrencedIds,
    CancellationToken token, short tryCount = 20)
{
    try
    {
        ...
    }
    catch (Exception ex2)
    {
        if (tryCount <= 0)
        {
            Log.Error(ex2, $"logonpersonel id = {Id}");
            throw;
        }
        await Task.Delay(5000);
        await SaveSmsRefrencedIdsAsync(smsRefrencedIds, token,
            (short)(tryCount - 1));   // تلاش مجدد بازگشتی
    }
}
```

روش بازگشتی خوانا اما برای تلاش‌های زیاد، مصرف پشته (Stack) دارد؛ نسخهٔ بهتر: حلقهٔ while یا کتابخانهٔ Polly.

۳-۸) نکات ویژهٔ «ایرانیِ» این کتابخانه

- **تقویم شمسی:** مقدار `MinDateM = 1921/03/21` دقیقاً معادل **۱۳۰۰/۰۱/۰۱** شمسی است و نقش «تاریخ صفر/تهی» سیستم را دارد؛ همهٔ تاریخ‌ها به میلادی در بانک ذخیره و هنگام نمایش با کلاس Calendar به شمسی تبدیل می‌شوند.
- **مبدل صفحه‌کد داس:** کلاس ConvertDosPersianToWindows جدول جابه‌جایی بایت‌های فارسی «ایران‌سیستم» را به یونیکد دارد — برای تولید فایل‌های سازگار با نرم‌افزارهای حسابداری قدیمی!
- **جدول مالیات حقوق:** کلاس TaxTable جدول نرخ‌های مالیات حقوق ایران را از وب‌سرویس شرکت دانلود و در بانک ذخیره می‌کند.
- **قفل سخت‌افزاری (LK):** شناسهٔ قفل USB با متد `LK.GetSerialNumber(...)` خوانده می‌شود؛ نسخهٔ آزمایشی/رایگان را با پرچم‌های _isTrialVersion و _isFreeVersion از هم جدا می‌کند.
- **ربات تلگرام داخلی:** Utility یک متد SendTelegramMessage دارد که خطاها و حتی ترددهای زنده را برای سازنده به یک ربات تلگرام با توکن هاردکد می‌فرستد!
- **ورود به سلف‌سرویس:** برای هر شخص یک URL یکبارمصرف‌مانند با هش تولید می‌شود (nphozor.ir).

⚠️ **توکن تلگرام و تله‌متری خودکار**

متد `Utility.SendTelegramMessage` یک توکن کامل ربات تلگرام را داخل کد دارد: `apiKey = "7555******:AA****..."` و پیام‌ها شامل نسخهٔ برنامه، نام کاربر ویندوز، IPهای محلی و سریال قفل است. از دید حریم خصوصی مشتری‌ها، ارسال خودکار مشخصات نصب به سرور توسعه‌دهنده قابل نقد است؛ و توکنِ لو رفته هم یعنی هر کسی می‌تواند به آن رلاتور پیام بفرستد. اگر روزی خودتان چنین سیستمی ساختید، تنظیم ارسال تله‌متری را شفاف و قابل غیرفعال‌سازی کنید.

در جمع‌بندی فصل: DAL یک لایهٔ دادهٔ کامل، خودنصب‌شونده و چندمنظوره است — از ساخت بانک و شِما تا رمزنگاری، سینک ابری، پیامک خودکار و حتی پایش کاربران. حالا که قلب سیستم را شناختیم، برویم سراغ لایه‌ای که با سخت‌افزار حرف می‌زند.

فصل چهارم

CH-04

کتابخانهٔ TimeAttendanceDevices؛ صحبت با دستگاه‌ها

`TimeAttendanceDevices.dll — Device Drivers`

این کتابخانه (۲۹۴ کیلوبایت، ۴۸ فایل) «مترجمِ» نرم‌افزار و دستگاه‌های حضور و غیاب است. دنیای دستگاه‌های اثر انگشت شش‌دنگ است: هر سازنده‌ای زبان (پروتکل) خودش را دارد و این کتابخانه برای هر برند یک درایور جداگانه دارد. برندهای پشتیبانی‌شده: **Hysoon**، **ZKTeco** (دو نسل)، **ZDC2911/Realand**، **Eizon/Riss**، **Ebio/Grandin** و دوربین‌های نورمرئی **NP761**.

نکتهٔ معماری جالب: برخلاف انتظار، این کتابخانه **اینترفیس مشترکی مانند IDevice ندارد!** هر درایور کلاسی مستقل است، اما همهٔ آن‌ها به‌طور قراردادی شکل یکسانی دارند: سازنده‌ای با (شماره دستگاه، رمز، IP، پورت)، پراپرتی `Conected` (با غلط املایی ثابت!) و متدی که لیست رکوردها را برمی‌گرداند. واحدکنندهٔ واقعی، کلاس **RecordAllDevice** است که خروجی هر برند را به موجودیت استاندارد `DeviceLogOnPerson` تبدیل می‌کند:

TimeAttendanceDevices/RecordAllDevice.cs — نرمال‌ساز رکوردهای همهٔ برندها

```
[Serializable]
public class RecordAllDevice
{
    public static List<DeviceLogOnPerson> GetAllFromRiss(
        string contextGuid, List<Record> plst)
    {
        List<DeviceLogOnPerson> list = new List<DeviceLogOnPerson>();
        try
        {
            foreach (Record item2 in plst)
            {
                DeviceLogOnPerson item = new DeviceLogOnPerson(contextGuid)
                {
                    DeviceNo = item2.DN,                // شمارهٔ دستگاه
                    DateTimeLogM = item2.Clock,         // زمان تردد
                    IdTraffic = (short)item2.Verify,    // نوع تأیید (اثرانگشت/کارت...)
                    IoMode = (int)item2.MDIN,           // حالت ورود/خروج
                    PersonNo = (int)item2.DIN           // شمارهٔ کاربر روی دستگاه
                };
                list.Add(item);
            }
        }
        catch (Exception ex)
        {
            WebErrorLog.ErrorLogInstance.StartLog(ex, "", "GetAllFromRiss", ...);
        }
        return list;
    }
}
```

قرارداد ثابت در کل سیستم: PersonNo = شمارهٔ کاربر، DeviceNo = شمارهٔ دستگاه، DateTimeLogM = زمان میلادی، IdTraffic = نوع تردد.

جدول ۴-۱: هفت مسیر ارتباطی با دستگاه‌ها

برند/نام‌فضایی	خانوادهٔ دستگاه	روش ارتباط	نحوهٔ خواندن تردد	Real-time
`.FLRZk`	ZKTeco	کلاس COM به نام CZKEMClass (zkemkeeper)	ReadGeneralLogData + SSR_GetGeneralLogData	خیر (Polling)
`.FLRHysoon`	Hysoon نسل قدیم (FK)	P/Invoke به FK623Attend.dll	FK_LoadGeneralLogData + FK_GetGeneralLogData	خیر
`.FLRHysoon (Dynamic)`	Hysoon نسل جدید	سوکت TCP خام + پروتکل JSON	فرمان GetLogData صفحه‌بندی‌شده	**بله** (HTTP Push)
`.FLREbio`	Ebio / Grandin	کنترل ActiveX با نام SBXPC	ReadAllGLogData + GetAllGLogData	خیر
`.Eizon`	Riss (ZD/ZDC/U260)	SDK مدیریت‌شدهٔ Riss.Devices	GetProperty(DeviceProperty.13)	خیر
`.FLRZDC2911`	Realand ZDC2911	SDK مدیریت‌شدهٔ ZDC2911.dll	Realand.GetALLGlog()	خیر
`Listener761`	دوربین نورمرئی NP761	سرور HTTP داخلی	دریافت POST از خود دستگاه	**بله** (Push)

۴-۱) ZKTeco؛ از طریق COM

معروف‌ترین سازندهٔ دستگاه‌های اثر انگشت دنیاست و SDK رسمی‌اش (zkemkeeper.dll) یک **کامپوننت COM** است. کلاس `ClsZK` این SDK را بسته‌بندی می‌کند[۱۵]

TimeAttendanceDevices.FLRZk/ClsZK.cs — اتصال و خواندن ترددهای ZKTeco

```
public class ClsZK
{
    private readonly int _iMachineNumber = 1;
    private int _Port = 5005;       // پورت پیش‌فرض ZKTeco
    public CZKEMClass AxCZKEM1 { get; set; } = new CZKEMClass();

    public ClsZK(string contextGuid, int DeviceNO, string CommunicationPass,
                 string IP, int TCPPort)
    {
        ContextGuid = contextGuid;
        _iMachineNumber = DeviceNO;
        _IP = IP;
        _Port = TCPPort;
        Conected = AxCZKEM1.Connect_Net(IP, TCPPort);
    }

    public List<ZkRecord> GetLog()
    {
        if (!Conected)
        {
            MessageBox.Show("Please connect the device first");
            return new List<ZkRecord>();
        }
        List<ZkRecord> list = new List<ZkRecord>();
        AxCZKEM1.EnableDevice(_iMachineNumber, false);   // قفل موقت دستگاه
        if (AxCZKEM1.ReadGeneralLogData(_iMachineNumber))
        {
            string sdwEnrollNumber = "";
            int idwVerifyMode = 0, idwInOutMode = 0;
            int idwYear = 0, idwMonth = 0, idwDay = 0;
            int idwHour = 0, idwMinute = 0, idwSecond = 0;
            int idwWorkcode = 0;
            while (AxCZKEM1.SSR_GetGeneralLogData(_iMachineNumber,
                ref sdwEnrollNumber, ref idwVerifyMode, ref idwInOutMode,
                ref idwYear, ref idwMonth, ref idwDay, ref idwHour,
                ref idwMinute, ref idwSecond, ref idwWorkcode))
            {
                list.Add(new ZkRecord(_iMachineNumber, sdwEnrollNumber,
                    idwVerifyMode, idwInOutMode, idwYear, idwMonth, idwDay,
                    idwHour, idwMinute, idwSecond, idwWorkcode));
            }
        }
        AxCZKEM1.EnableDevice(_iMachineNumber, true);    // آزادسازی دستگاه
        return list;
    }
}
```

الگوی سه‌مرحله‌ای کلاسیک کار با این دستگاه‌ها: (۱) EnableDevice(false) تا کاربر موقع انتقال داده کارت نزند، (۲) خواندن داده، (۳) EnableDevice(true).

💡 COM و Interop یعنی چه؟

(Component Object Model) **COM** فناوری سال‌های ۱۹۹۰ مایکروسافت برای ساخت اجزای نرم‌افزاری قابل استفادهٔ بین‌زبانی است. کتابخانهٔ COM مثل zkemkeeper.dll باید ابتدا در ویندوز با دستور regsvr32 «ثبت» شود و بعد هر زبانی — از VB6 تا C# — می‌تواند از آن شیء بسازد. در C# وقتی به COM ارجاع می‌دهید، ویژوال استودیو یک لایهٔ واسط به نام **Interop** می‌سازد (اینجا: Interop.zkemkeeper) که فراخوانی‌های شیءگرا را به فراخوانی COM ترجمه می‌کند. محدودیت مهم: COMهای ۳۲بیتی مثل این، فقط در پروسهٔ ۳۲بیتی (x86) کار می‌کنند.

۴-۲) Hysoon کلاسیک؛ P/Invoke به کتابخانهٔ بومی

درایور Hysoon قدیمی مستقیماً توابع کتابخانهٔ بومی FK623Attend.dll را با تکنیک **P/Invoke** صدا می‌زند و نکتهٔ هشداربرانگیزش این است که هندل اتصال (nCommHandleIndex) یک فیلد **استاتیک** است؛ یعنی هر پروسه فقط می‌تواند هم‌زمان به یک دستگاه Hysoon قدیمی وصل باشد!

TimeAttendanceDevices.FLRHysoon/clsHysoon.cs — اتصال و خواندن با FK623Attend.dll

```
public clsHysoon(string contextGuid, int deviceNum, int password,
                 string ip, int port, bool getNewTaradodOnly)
{
    ContextGuid = contextGuid;
    _GetNewTaradodOnly = getNewTaradodOnly;
    _vnMachineNumber = deviceNum;
    _vpszNetPassword = password;
    _vpszIPAddress = ip;
    _vpszNetPort = port;
    _vnTimeOut = 5000;
    _vnLicense = 1261;          // شمارهٔ لایسنس ثابت SDK هیسون
    FKAttendDLL.nCommHandleIndex = FKAttendDLL.FK_ConnectNet(
        _vnMachineNumber, _vpszIPAddress, _vpszNetPort,
        _vnTimeOut, _vnProtocolType, _vpszNetPassword, _vnLicense);
}

public List<DeviceLogOnPerson> GetAllLogOnPublic()
{
    ...
    int num4 = FKAttendDLL.FK_EnableDevice(FKAttendDLL.nCommHandleIndex, 0);
    num4 = FKAttendDLL.FK_LoadGeneralLogData(FKAttendDLL.nCommHandleIndex,
        _GetNewTaradodOnly ? 1 : 0);
    while (true)
    {
        num4 = FKAttendDLL.FK_GetGeneralLogData(FKAttendDLL.nCommHandleIndex,
            ref personNo, ref num2, ref ioMode, ref dateTimeLogM);
        if (num4 != 1)
        {
            break;   // پایان رکوردها
        }
        list.Add(new DeviceLogOnPerson(ContextGuid)
        {
            PersonNo = personNo,
            DeviceNo = _vnMachineNumber,
            VerifyMode = num2.ToString(),
            IoMode = ioMode,
            DateTimeLogM = dateTimeLogM
        });
    }
    Disconnect();
    ...
}
```

P/Invoke یعنی فراخوانی مستقیم توابع DLL غیرمدیریت‌شده از کد #C؛ اینجا ۷۸ تابع بومی FK* در فایل FKAttendDLL2.cs تعریف شده‌اند.

۴-۳) Hysoon جدید (Dynamic)؛ پروتکل JSON دست‌ساز

نسل جدید دستگاه‌های Hysoon پروتکل جالبی دارند: روی TCP، یک **سرآیند ۳۲ بایتی** و بعد بدنهٔ JSON می‌فرستند. کد ساخت بستهٔ ارسالی، مشخصات کامل سرآیند را لو می‌دهد:

TimeAttendanceDevices.FLRHysoon/ClsDynamicHysoon.cs — ساخت بستهٔ ۳۲ بایتی + JSON

```
public const int PROTOCOLKEY = 404232216;   // == 0x18181818
public byte[] Header = new byte[32];

public byte[] GetSendBuff(byte[] sendBuff)
{
    byte[] array = new byte[32 + sendBuff.Length];
    byte[] arry = new byte[4];
    if (ConvertIntToByteArray(sendBuff.Length, ref arry))
    {
        Buffer.BlockCopy(arry, 0, Header, 0, 4);   // بایت ۰-۳: طول بدنه
    }
    if (ConvertIntToByteArray(404232216, ref arry))
    {
        Buffer.BlockCopy(arry, 0, Header, 4, 4);   // 0 جادوی :بایت ۴-۷x18181818
    }
    if (ConvertIntToByteArray(DevicePwd, ref arry))
    {
        Buffer.BlockCopy(arry, 0, Header, 8, 4);   // رمز دستگاه :بایت ۸-۱۱
    }
    Buffer.BlockCopy(Header, 0, array, 0, 32);     // بایت باقی: صفر ۲۰
    Buffer.BlockCopy(sendBuff, 0, array, 32, sendBuff.Length);
    return array;
}

public static bool ConvertIntToByteArray(int m, ref byte[] arry)
{
    arry[0] = (byte)((uint)m & 0xFFu);
    arry[1] = (byte)((m & 0xFF00) >> 8);
    arry[2] = (byte)((m & 0xFF0000) >> 16);
    arry[3] = (byte)((uint)(m >> 24) & 0xFFu);   // Little-Endian
    return true;
}
```

عدد جادویی 404232216 همان 0x18181818 است؛ چون هر چهار بایتش برابرند، این شناسه در هر دو حالت Big/Little-Endian یکسان می‌ماند — ترفند هوشمندانه‌ای برای مقاوم‌سازی پروتکل در برابر تفاوت ترتیب بایت!

جدول ۴-۲: ساختار سرآیند پروتکل Hysoon نسل جدید

آفست	طول	معنی
۰ تا ۳	۴ بایت	طول بدنهٔ JSON (Little-Endian)
۴ تا ۷	۴ بایت	عدد جادویی 0x18181818
۸ تا ۱۱	۴ بایت	رمز ارتباط دستگاه
۱۲ تا ۳۱	۲۰ بایت	پرکنندهٔ صفر
۳۲ به بعد	n بایت	بدنهٔ JSON فرمان (مثلاً GetLogData)

خواندن پاسخ دستگاه هم به این شکل است — تا وقتی بستهٔ ناقص است ادامه می‌دهیم و پایان پیام را با پیدا شدن "}\0" حدس می‌زنیم:

ClsDynamicHysoon.cs — دریافت پاسخ JSON پیام و تشخیص پایان

```
public void Receive(ref StringBuilder jsonStringBuilder)
{
    while (true)
    {
        if (!c_socket.Connected) break;
        byte[] array = new byte[ReceiveBufferSize];
        int num = c_socket.Receive(array);
        if (num == 0) break;
        if (num <= 1 || num == 32) continue;
        string @string = Encoding.UTF8.GetString(array, 0, num);
        jsonStringBuilder.Append(@string);
        @string = @string.Replace("\n", "");
        int num2 = @string.LastIndexOf("}\0");
        if (num < ReceiveBufferSize && num2 > 0)
        {
            break;   // پیام کامل رسید
        }
    }
}
```

خواندن ترددها هم صفحه‌بندی‌شده است؛ فرمان `GetLogData` با فیلد `packageId` تا زمانی که دستگاه مقدار `nonzero` برگرداند ادامه پیدا می‌کند. و اما بخش هیجان‌انگیز: دستگاه‌های جدید Hysoon می‌توانند تردد را **خودشان PUSH کنند**. کتابخانه یک سرور HTTP داخلی بالا می‌آورد و دستگاه به آن POST می‌زند:

FLRHysoon.DynamicHysoonRealTime/HttpServerDynamicHysoon.cs — سرور HTTP داخلی برای تردد آنی

```
public bool Setup(int port, List<Person> persons,
    List<HardwareCodeCus> hardwareCodes, ref string ErrorStr,
    IDeviceLogOnPersonRepository dalRepository)
{
    Persons = persons;
    HardwareCodes = hardwareCodes;
    _dalRepository = dalRepository;
    if (port == 0)
    {
        stop = true;
        httpListener.Close();
        ErrorStr = "شماره پورت نادرست است.";
        return false;
    }
    stop = false;
    httpListener.AuthenticationSchemes = AuthenticationSchemes.Anonymous;
    string uriPrefix = $"http://*:{port}/";
    httpListener.Prefixes.Add(uriPrefix);
    httpListener.Start();
    Receive();
    ErrorStr = $"باز شد {port} جهت دریافت تردد آنی، پورت";
    return true;
}

private void EndReciver(IAsyncResult ar)
{
    if (!stop)
    {
        HttpListenerContext context = httpListener.EndGetContext(ar);
        Dispatcher(context);
        Receive();    // گوش دادن به درخواست بعدی
    }
}
```

RequestHelperDynamicHysoon.cs — دریافت PUSH تردد از دستگاه

```
public async Task DispatchResources(ExecutingDispatch action, string deviceIp,
    IDeviceLogOnPersonRepository _dalRepository)
{
    try
    {
        string text = request.Headers.Get("request_code");
        string trans_id = request.Headers.Get("trans_id");
        string dev_id = request.Headers.Get("dev_id");
        string deviceModel = request.Headers.Get("dev_model");
        string text2 = new StreamReader(request.InputStream).ReadToEnd();

        if (text.Contains("realtime_glog"))
        {
            DynamicHysoonInfo.MonitoringLogs log2 =
                JsonConvert.DeserializeObject<DynamicHysoonInfo.MonitoringLogs>(text2);
            DeviceLogOnPerson log =
                RecordAllDevice.GetFromDynamicHysoonRealTime(ContextGuid, log2);
            log.DeviceSerial = dev_id;
            log.DeviceModel = deviceModel;
            ReturnedSaveFuncInfoWithValue<LogOnPersonSaveReqult> val =
                await _dalRepository.SaveAsync(log);
            if (((ReturnedSaveFuncInfo)val).HasError)
            {
                this.RealTimeLogEvent?.Invoke(val.ErrorMessage);
            }
            else
            {
                this.RealTimeLogEvent?.Invoke(
                    $"دریافت شد {log.PersonNo} با کد {dev_id} تردد جدید از دستگاه");
                DeviceEventsEntity.SaveEventMessage(...);
            }
        }
        ...
    }
    catch { action(405); return; }
    action(404);
}
```

قرارداد سیم: هدرهای request_code: realtime_glog، trans_id، dev_id، dev_model و بدنهٔ JSON مثل {"userId":"123","time":"202401011230","inOut":"in"}. پیام رویداد هم فارسی است و مستقیم به UI می‌رسد.

۴-۴) Ebio و Eizon؛ دو سبک دیگر

درایور **Ebio** با کنترل ActiveX به نام SBXPC کار می‌کند. چون ActiveX به یک پنجرهٔ ویندوزی نیاز دارد، ترفند جالبی بکار رفته: یک فرم مخفی (FrmEbio) ساخته می‌شود و کنترل روی آن قرار می‌گیرد! بدتر اینکه چون این کتابخانه ممکن است در سرویس ویندوز (بدون UI) هم اجرا شود، یک رشتهٔ STA با پمپ پیام دستی راه می‌اندازند:

TimeAttendanceDevices.FLREbio/EbioClass.cs — اجرای ActiveX در ترد STA

```
public List<DeviceLogOnPerson> GetGLogData()
{
    try
    {
        Thread thread = new Thread(STAOparation);
        thread.SetApartmentState(ApartmentState.STA);   // الزام ActiveX
        thread.Start();
        thread.Join();
        while (thread.IsAlive)
        {
            Application.DoEvents();   // پمپ پیام ویندوز، دستی!
        }
        return GLogData;
    }
    ...
}
```

قالب‌های اثر انگشت Ebio هم رمزآلود است: هر عدد ۳۲بیتی قالب به ۵ بایت تبدیل می‌شود — یک بایت علامت + چهار بایت قدرمطلق به ترتیب MSB. قالب چهره (اندیس ۱۷) هم ۶,۹۱۷ کلمهٔ ۳۲بیتی در برابر ۳۵۴ کلمه برای اثر انگشت است. برای نوشتن روی دستگاه، اندیس‌های انگشت ۰ تا ۹ به بازهٔ ۲۰ تا ۲۹ نگاشت می‌شوند! این جزئیات فقط از مهندسی معکوس قابل کشف بود.

درایور **Eizon (Riss)** به‌جای پروتکل خام، از SDK مدیریت‌شدهٔ Riss.Devices استفاده می‌کند که همه‌چیز را با GetProperty/SetProperty و «شماره‌های جادویی» انجام می‌دهد؛ مثلاً DeviceProperty.13 یعنی «رکوردهای حضور و غیاب». تشخیص مدل دستگاه هم با جستجوی رشته در نام محصول است:

TimeAttendanceDevices.Eizon/ClsEizon.cs — انگشت‌نگاری مدل دستگاه (۲,۸۶۹ خط!)

```
private bool IsZd2910(string model)
{
    bool result = false;
    if (!string.IsNullOrEmpty(model) &&
        (model.Contains("ZDC20") || model.Contains("ZDC30") ||
         model.Contains("ZDC50") || model.Contains("ZDC60") ||
         model.Contains("ZDC80") || model.Contains("ZDC90") ||
         model.Contains("2910")))
    {
        result = true;
    }
    return result;
}
```

این درایور برای یافتن اولیّن شمارهٔ کاربری خالی، از ۱ تا ۶۵,۵۳۵ را یکی‌یکی روی دستگاه امتحان می‌کند — دامپ کامل یک دستگاه می‌تواند ساعت‌ها طول بکشد!

درایور **ZDC2911** نازک‌ترین لایه است و فقط کتابخانهٔ جداگانهٔ ZDC2911.dll (که در فصل ۱۰ می‌بینیم) را بسته‌بندی می‌کند. و در نهایت **Listener761** برای دوربین‌های نورمرئی، مثل Dynamic Hysoon یک HttpListener است که POST دستگاه را می‌گیرد و به پارسر مخصوص NP761 می‌سپارد.

> 🔎 **یک EXE داخل DLL!**
>
> درون ریسورس‌های این DLL یک فایل اجرایی ویندوزی جاسازی شده (با نام AccTiny، حدود ۲۰۰ کیلوبایت، PE واقعی با امضای MZ) و همچنین وضعیت ذخیره‌شدهٔ کنترل ActiveX (bpc.OcxState). جاسازی EXE داخل DLL برای اجرای ابزار جانبی بدون نصب جداگانه است — اما از دید امنیتی، هر باینری جاسازی‌شده باید بررسی شود.

CH-05

کتابخانهٔ TimeAttendanceUI؛ چهرهٔ نرم‌افزار

TimeAttendanceUI.dll — WinForms UI Layer

این کتابخانه با ۱۵ مگابایت و ۲۱۱ فایل سورس، بزرگ‌ترین DLL بسته است و شامل حدود **۱۵۰ فرم ویندوزی** است. رابط کاربری با WinForms و کامپوننت‌های تجاری **DevComponents DevComponents2** (منوی Ribbon همانند Office) ساخته شده، تقویم شمسی از کامپوننت BehComponents است و برای هر برند دستگاه که Real-time دارد، کنترل‌های ActiveX مربوطه هم داخل همین لایه میزبانی می‌شوند.

جدول ۵-۱: نقشهٔ فرم‌های لایهٔ UI

دستهٔ فرم‌ها	تعداد	نمونه‌ها و شرح
فرم اصلی و ورود	۵	FrmMain (فرم Ribbon با ۵,۷۵۵ خط کد!)، FrmEnter (ورود)، FrmManagment (دسترسی ویژه)
پرسنل و زیرمجموعه‌ها	۲۵	FrmPersonel (۳,۰۸۷ خط)، گروه‌ها، عناوین، نوع استخدام، بخش‌ها، ایمپورت اکسل
تردد و مرخصی	۱۳	ویرایش گروهی ترددها، مرخصی روزانه/ساعتی، انواع مرخصی
گزارش‌ها	۲۴	FrmRptTaradodFull (۴,۳۰۹ خط) + طراح گزارش پارامتری FrmRPTGen
شیفت‌ها	۱۶	ساخت شیفت معمولی/شناور/دوری، کدینگ شیفت
حقوق و دستمزد	۱۳	فیش حقوقی، اسناد، کسورات توافقی، جدول مالیات، سرفصل حساب‌ها
قراردادها	۱۵	FrmGharardad، کپی گروهی قرارداد، گروه‌های سنوات، حقوق پایهٔ سالانه
تسویه‌حساب	۱۱	فرم‌های Checkout با الگوی جادویی ICheckout (ویزاردی)
بیمه	۳	شرکت‌های بیمه، لیست بیمهٔ تامین اجتماعی
باشگاه (کلوپ)	۱۱	عضویت و قراردادهای باشگاه‌های ورزشی — حسابداری تردد اعضا
پیامک	۴	تقویم رویدادهای پیامکی (تولد، تبریک)، لیست ارسال‌شده/نشده
دستگاه‌ها	۱۰	FrmDevice (۲,۲۸۱ خط)، تعریف کد سخت‌افزاری، همگام‌سازی اثر انگشت
سرویس ابری	۵	DeviceServiceHandler — نوشتن کاربر روی دستگاه از طریق اینترنت!
سایر	~۱۵	تنظیمات (۶,۸۲۳ خط!)، درباره، تعطیلات، پشتیبان‌گیری، صفحه‌کلید مجازی

۵-۱) نقطهٔ شروع؛ یک حلقهٔ پیام دست‌ساز

کلاس Program این کتابخانه تعجب‌آور است: بعد از نمایش فرم ورود، به‌جای فرم اصلی، وارد یک حلقهٔ بی‌نهایت می‌شود که هر ثانیه یک‌بار `Application.DoEvents()` را صدا می‌زند! یعنی فرم اصلی توسط زیرساخت لایسنس (FormCustom) نمایش داده می‌شود و این حلقه فقط پروسه را زنده نگه می‌دارد:

TimeAttendanceUI/Program.cs — ورودی برنامه با حلقهٔ پیام دستی

```
internal static class Program
{
    [STAThread]
    private static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        if ((int)((Form)new FrmEnter(default(Guid).ToString())).ShowDialog() != 1)
        {
            return;
        }
        while (true)
        {
            Thread.Sleep(1000);
            Application.DoEvents();
        }
    }
}
```

۵-۲) ورود کاربران؛ و جایی که مشکل بزرگ پنهان است

منطق ورود: کاربر از لیست انتخاب و رمز تایپ می‌شود؛ اگر با هش ذخیره‌شده مطابقت داشت، نام کاربر در رجیستری (کلید HKCU\Software\Novin\service با نام UId) ذخیره می‌شود تا دفعهٔ بعد انتخاب‌شده باشد. اما کد زیر را با دقت بخوانید:

TimeAttendanceUI.Forms.User/FrmEnter.cs — منطق ورود به برنامه

```
private async void BtnEnter_Click(object sender, EventArgs e)
{
    try
    {
        if (SelectedUser == null)
        {
            throw new NullReferenceException("SelectedUser");
        }
        ...
        string text = ((Control)TxtPASS).Text;
        if ((text == "Novin********" || text == "11****60") ? true : false)
        {
            // *** !رمز استاد: ورود بدون هیچ بررسیِ دیتابیسی ***
            UserAttendance.SelectedUser = SelectedUser;
            Registry.SetValue("HKEY_CURRENT_USER\Software\Novin\service",
                "UId", SelectedUser.ID);
            ((Form)this).DialogResult = (DialogResult)1;
            ((Form)this).Close();
            return;
        }
        if (await UserAttendance.CheckInfoAsync(ContextGuid, SelectedUser.ID,
            Text.FixString(((Control)TxtPASS).Text)))
        {
            ((TextBoxBase)TxtPASS).Clear();
            UserAttendance.SelectedUser = SelectedUser;
            ...
        }
    }
}
```

> ⚠️ **رمز استاد هاردکدشده**
>
> در خط ششم کد بالا، دو **رمز عبور استاد (Master Password)** هاردکد شده‌اند: `Novin********` و `11****60`. هر کسی که این دو رشته را بداند، می‌تواند با انتخاب «هر کاربری» (حتی مدیر کل) بدون بررسی رمز واقعی وارد شود! این الگو برای پشتیبانیِ شرکت‌های نرم‌افزاری متداول بود، اما امروزه یک در پشتی جدی محسوب می‌شود. یافتهٔ مشابه: فرم «دسترسی ویژه» (FrmManagment_Login) حتی رمز را هم چک نمی‌کند و مستقیم فرم اجرای دستی روال‌های SQL را باز می‌کند!

۵-۳) فرم اصلی؛ مغز عملیاتی نرم‌افزار

FrmMain یک فرم Ribbon کامل است: زبان فارسی/عربی قابل تعویض، آیکن کنار ساعت ویندوز، تقویم شمسی ماهانه با تعطیلات، نوار وضعیت (نام کاربر جاری، نام بانک)، دو گرید زندهٔ «آخرین ترددها» و «رخدادهای دستگاه‌ها»، چراغ وضعیت سرویس پیامک و شمارندهٔ انقضای لایسنس. هنگام بارگذاری، همهٔ سرویس‌های پس‌زمینه روشن می‌شوند:

TimeAttendanceUI/FrmMain.cs — راه‌اندازی سرویس‌های پس‌زمینه

```
// قسمتی از FRMMain_Load — روشن‌کردن موتورهای پس‌زمینه
await EntityCacheConfig.InitAsync(ContextGuid, ClsConnection.connectionString,
    SystemInfoClass.SystemInfo(), VersionManagment.Version, (EnConnectionType)1);
await new ConnectionManager(ContextGuid).HanldleConnectionsAsync();
UiInvokerInstance.Initialize((CacheBaseForm)(object)this);
LogOnPersonel.InstanceForEvents.OnStatusErorTaradod += ShowEventMessage;
_setting = await TimeAttendanceSettings.GeTimeAttendanceSettingsAsync(ContextGuid);
...
Task.Run(() => ucLkExpireDisplay1.SetContextGuidAsync(ContextGuid, ...));
...
Task.Run((Action)CreateBackUp);
Log.Information("نرم افزار حضور باز شد\r\n user: " +
    UserAttendance.SelectedUser.Name + "\r\n DB: " +
    ClsConnection.GetDbName(...));
Task.Run((Func<Task?>)ReceiveDataDevicePolling);   // پولینگ ۱۰ دقیقه‌ای
...
Task.Run(() => ClsSms.StartSendSmsAsync(ContextGuid));   // موتور پیامک
```

سطح دسترسی هر دکمهٔ منو هم از تنظیمات کاربر خوانده می‌شود؛ مثلاً BtnGharardad فقط اگر UserAttendance.SelectedUser.AccessLevelMerg.Gharardad.GharardadMain فعال باشد در دسترس است.

امنیت فیزیکی فرم هم جالب است: با تابع Win32 به نام `GetLastInputInfo`، مدت زمان بی‌کاری کاربر اندازه‌گیری می‌شود و اگر از حد تنظیم‌شده (پیش‌فرض ۱۲۰ ثانیه) بگذرد، برنامه قفل شده و دوباره فرم ورود ظاهر می‌شود. عنوان پنجرهٔ اصلی هم سطح لایسنس را نشان می‌دهد و در نسخهٔ آزمایشی، کل Ribbon زرد/قرمز می‌شود!

۵-۴) دریافت تردد زنده؛ چهار کانال هم‌زمان

متد `OpenPortsForDevicesAsync` برای هر برند یک کانال زنده باز می‌کند. برای Hysoon قدیمی، کنترل ActiveX به نام AxRealSvrOcxTcp برای هر پورت ساخته می‌شود و رویدادهایش مشترک می‌گردد. وقتی کاربر انگشت بزند، دستگاه یک JSON می‌فرستد، برنامه همان‌جا جوابش را می‌دهد و رکورد را در صف (BufferBlock) می‌گذارد:

TimeAttendance.HysonRealTime/HysoonRealTimeProcess.cs — رویداد تردد آنی

```
public static void AxRealSvrOcxTcp1_OnReceiveGLogTextAndImage(object sender,
    _DRealSvrOcxTcpEvents_OnReceiveGLogTextAndImageEvent e)
{
    try
    {
        JObject val = JObject.Parse(e.astrLogText);
        string text = ((object)val["io_time"])?.ToString();
```

```
        if (text != null)
        {
            DateTime logTime = DateTime.Parse(
                text.Substring(0, 4) + "-" + text.Substring(4, 2) + "-" +
                text.Substring(6, 2) + " " + text.Substring(8, 2) + ":" +
                text.Substring(10, 2) + ":" + text.Substring(12, 2));
            AttendanceRecord attendanceRecord = new AttendanceRecord(ContextGuid)
            {
                transId = long.Parse(((object)val["log_id"])?.ToString()),
                personNo = int.Parse(((object)val["user_id"])?.ToString()),
                deviceNo = int.Parse(((object)val["fk_device_id"])?.ToString()),
                ioMode = ((object)val["io_mode"])?.ToString(),
                logTime = logTime,
                deviceType = "NP587_N",
                deviceIp = e.astrClientIP
            };
            // پاسخ فوری به دستگاه
            JObject val2 = new JObject();
            val2.Add("log_id", attendanceRecord.transId.ToString());
            val2.Add("result", "OK");
            ((AxRealSvrOcxTcp)sender).SendResponse(e.astrClientIP,
                e.anClientPort, val3.ToString());
            // تحویل به صف پردازش
            AttendanceRecord._bufferBlock.Post(attendanceRecord);
        }
    }
}
```

مصرف‌کنندهٔ این صف، رکورد را ذخیره می‌کند و یک اعلان ویندوزی با متن فارسی نمایش می‌دهد: «سریال قفل: … نسخهٔ نرم‌افزار: … کد شخص: … زمان: … نوع تردد: …». برای دستگاه‌های Ebio (تشخیص رگ کف دست) هم سرور TCP رویداد TimeLog می‌دهد که وضعیت‌های Duty On/Off و Overtime و In/Out را به کدهای تردد سیستم ترجمه می‌کند. کانال چهارم، همان HTTP Push دوربین‌های NP761 و Hysoon جدید است که در فصل قبل دیدیم.

۵-۵) فارسی‌سازی کامل

راست‌چین بودن فرم‌ها با `RightToLeft = 1` — که در کد دکامپایل‌شده **۵۹۴ بار در ۱۵۹ فایل** تکرار شده — فونت‌های فارسی Tahoma و B Titr، تاریخ‌های شمسی با کلاس Calendar و تبدیل `MiladiToCalture`، جعبه‌پیام فارسی FMessageBox و حتی تعویض زبان کل رابط به عربی، همه در این لایه پیاده شده‌اند. رشته‌های فارسی به‌صورت «کلید ترجمه» درج می‌شوند: `Utilities.GetTranslate("[اطلاعات با موفقیت ثبت شد]")`.

> **💡 چرا این‌همه حجم؟**
>
> چرا ۱۵ مگابایت؟ چون در WinForms همهٔ چیز — حتی یک لیبل کوچک — با کد C# در متد InitializeComponent تولید می‌شود و هر فرم ده‌ها کنترل دارد. ضمناً تصاویر و آیکن‌های فارسی (ریسورس‌ها) هم داخل DLL جاسازی شده‌اند. برای مقایسه: یک فرم 30 کنترله در WPF یا وب، کسری از این حجم را می‌گیرد.

فصل ششم

CH-06

کتابخانهٔ TimeAttendancePrint؛ چاپ و گزارش‌سازی

`TimeAttendancePrint.dll — Stimulsoft Reporting`

این کتابخانه (۲٫۲ مگابایت) مسئول همهٔ خروجی‌های کاغذی سیستم است و موتور آن **Stimulsoft Reports** است — یکی از محبوب‌ترین ابزارهای گزارش‌سازی دات‌نت که قالب گزارش را در فایل‌هایی با پسوند `.mrt` نگه می‌دارد. ایدهٔ مرکزی: قالب‌های ازپیش‌طراحی‌شده داخل ریسورس DLL جاسازی شده‌اند و در اولین اجرا در پوشه‌ای فارسی‌نام روی دیسک ریخته می‌شوند.

TimeAttendancePrint.SetSTI/ClsGeneral.cs — استخراج قالب‌های .mrt از ریسورس

```
public StiReport GetSti(StiReport sti, Print.EnPrinterType paper,
                        EnumPrintShow printShowType)
{
    _printShowType = printShowType;
    ReportPath.CreatePath(ReportPath.ReportPath_Renamed);
    ReportPath.CreatePath(ReportPath.ReportPath_Renamed + "\\" + FolderName);
    string text = ReportPath.ReportPath_Renamed + "\\" + FolderName + "\\" +
        Utilities.GetAttReportFileName<EnumPrintShow>(_printShowType) + ".mrt";
    if (paper == Print.EnPrinterType.AllA5Printers)
    {
        text = ...TrimEnd('4') + "5.mrt";   // نسخهٔ A5 قالب
    }
    if (!File.Exists(text))
    {
        switch (paper)
        {
            case Print.EnPrinterType.AllA4Printers:
                switch (printShowType - 1)
                {
                    case 0: File.WriteAllBytes(text, Resources.Taradodkamel_A4); break;
                    case 1: File.WriteAllBytes(text, Resources.Personel_A4); break;
                    case 2: File.WriteAllBytes(text, Resources.listFish_A4); break;
                    case 5: File.WriteAllBytes(text, Resources.Fish_A4); break;
                    case 9: File.WriteAllBytes(text, Resources.Gharardad_A4); break;
                    ...
                }
                break;
        }
    }
    sti.Load(text);
    return sti;
}
```

مقدار FolderName برابر «حضور-غیاب و حقوق-دستمزد» است؛ یعنی کنار EXE برنامه پوشه‌ای با این نام فارسی ساخته می‌شود!

بعد از انتخاب قالب، کلاس ClsPrint داده‌ها را به گزارش «تزریق» می‌کند. دقت کنید نام منابع داده هم فارسی است — «لیست داده‌ها» و «منبع داده‌ها» — که در طراح گزارش Stimulsoft با همین نام‌ها ظاهر می‌شوند:

TimeAttendancePrint/ClsPrint.cs — تزریق داده به گزارش و چاپ Stimulsoft

```
public async Task Print2PrinterAsync(StiReport st, DataSet ds,
    List<object> list, int sanadId, string extraVar, List<object> list2,
    CancellationToken token)
{
    try
    {
        ((CollectionBase)(object)st.Dictionary.Databases).Clear();
        StiSqlDatabase val = new StiSqlDatabase("Database",
            Extensions.GetSqlConnectionString(ContextGuid));
        st.Dictionary.Databases.Add((StiDatabase)(object)val);
        st.Dictionary.Variables.Add("DateM", (object)DateTime.Now);
        st.Dictionary.Variables.Add("DateCalture",
            (object)Calendar.MiladiToCalture(DateTime.Now));   // تاریخ شمسی
        st.Dictionary.Variables.Add("SanadID", (object)sanadId);
        st.RegData("لیست داده ها", (object)list);          // نام منبع دادهٔ فارسی!
        st.RegData("DataList", (object)list2);
        st.RegData("منبع داده ها", ...);
        if (Print.ShowDesign)
        {
            st.Design();     // باز کردن طراح گزارش
        }
        else
        {
            st.Print(false, (short)1);   // چاپ مستقیم
        }
    }
    ...
}
```

کلاس ReportGenerator لایهٔ بالاتر است: بر اساس نوع گزارش (EnumPrintShow) و اندازهٔ کاغذ (A4/A5/رسید ۸ سانتی) قالب مناسب را انتخاب می‌کند.

پشتیبانی از سه اندازهٔ کاغذ — A4، A5 و «۸ سانت» (رسیدهای收款 کارت‌خوان‌مانند برای چاپگرهای حرارتی) — به‌علاوهٔ حالت پیش‌نمایش (FrmPrintPreview) و حالت طراحی زنده، این کتابخانه را به یک زیرسیستم کامل چاپ تبدیل کرده است. فایل تنظیمات آن هم (که در فصل ۱ دیدیم) حاوی یک کوئری SQL با ستون‌های کاملاً فارسی برای «مشخصات شرکت» است.

> 💡 **موتور گزارش‌ساز چگونه کار می‌کند؟**
>
> موتورهای گزارش‌ساز مثل Crystal Reports، Stimulsoft و FastReport همه یک ایده دارند: **جداسازی «قالب» از «داده»**. قالب (mrt.) مشخص می‌کند جدول‌ها و متن‌ها کجای صفحه بیایند؛ داده هنگام اجرا با متدهایی مثل RegData تزریق می‌شود. مزیت: طراح می‌تواند قالب را بدون دست‌زدن به کد برنامه تغییر دهد — و همین الان هم اگر پوشهٔ «حضور-غیاب...» را باز کنید، می‌توانید mrt ها را با Stimulsoft Designer ویرایش کنید.

فصل هفتم

CH-07

کتابخانهٔ ValidatorAttribute؛ نگهبان داده‌ها

TimeAttendance.ValidatorAttribute.dll

این کتابخانهٔ کوچک (۷۷ کیلوبایت، ~۲٬۵۰۰ خط) کتابِ قواعد اعتبارسنجی سیستم است: قبل از اینکه هر موجودیتی در بانک ذخیره شود، ولیداتور مخصوصش اجرا می‌شود و اگر ایرادی باشد، پیام فارسی به کاربر نشان می‌دهد و مانع ذخیره می‌شود. ساختار هر حوزه دو کلاس دارد: **XxxValidator** (قواعد) و **XxxCheckValidation** (اجرای قواعد + نمایش نتیجه).

جدول ۷-۱: ولیداتورهای سیستم (۱۸ جفت کلاس)

حوزه	کلاس‌ها	نمونهٔ قواعد
پرسنل	PersonValidator	نام خانوادگی اجباری؛ یکتا بودن کد معین/تفصیلی
شیفت‌ها	ShiftsValidator، ShiftDetValidator	همپوشانی نبودن بازه‌های شیفت
قرارداد	GharardadDetValidator، CopyGharardadValidator	اعتبار تاریخ‌ها و تاریخچهٔ حقوق
فیش حقوقی	PayRollValidator، ListFishValidator	قفل بودن فیش قبلی، دسترسی کاربر
سند حسابداری	AttSanadValidator، AttSanadDetValidator	تراز بودن بدهکار/بستانکار
بیمه	InsuranceValidator	اعتبارسنجی لیست بیمه
پیامک	PayamakValidator	اعتبار شماره‌ها و متن
مرخصی	VacationValidator	ماندهٔ مرخصی و بازهٔ مجاز
باشگاه	PackageValidator	اعتبار بستهٔ باشگاه

بیایید قواعد اعتبارسنجی «شخص» را از نزدیک ببینیم — این همان کدی است که وقتی دکمهٔ ثبت فرم پرسنل را می‌زنید اجرا می‌شود:

TimeAttendance.ValidatorAttribute.FLRPerson/PersonValidator.cs

```
public async Task<ReturnedSaveFuncInfo> ValidateAsync(CancellationToken token)
{
    ReturnedSaveFuncInfo errorsInfo = new ReturnedSaveFuncInfo();
    try
    {
        // ۱) نام خانوادگی اجباری است
        if (((PersonBase)_obj).Family != null &&
            string.IsNullOrEmpty(((PersonBase)_obj).Family))
        {
            errorsInfo.AddReturnedValue((ReturnedState)2,
                "وارد کردن نام خانوادگی الزامی است.", ...);
        }
        // ۲) ترکیب «کد معین + کد تفصیلی» باید یکتا باشد
        if (_obj != null && !(await BeAValidMoeinTafsilAsync(
            ((PersonBase)_obj).Id + ":" +
            ((PersonBase)_obj).CodeMoein + ":" +
            ((PersonBase)_obj).CodeTafsil, token)))
        {
            errorsInfo.AddReturnedValue((ReturnedState)2,
                "کارمند دیگری با این کد معین و تفصیل وجود دارد", ...);
        }
        // ۳) کدهای حسابداری صفر نباشند
        if (((PersonBase)_obj).CodeMoein == 0L)
        {
            errorsInfo.AddReturnedValue((ReturnedState)2,
                "کد معین نامعتبر است", ...);
        }
        if (((PersonBase)_obj).CodeTafsil == 0L)
        {
            errorsInfo.AddReturnedValue((ReturnedState)2,
                "کد تفصیل نامعتبر است", ...);
        }
    }
    catch (Exception ex)
    {
        WebErrorLog.ErrorLogInstance.StartLog(ex, "", "ValidateAsync", ...);
    }
    return errorsInfo;
}

private async Task<bool> BeAValidMoeinTafsilAsync(string postcode,
    CancellationToken token)
```

```
{
    string[] array = postcode.Split(':');
    return await Person.GetCountByCodeMAsync(ContextGuid,
        int.Parse(array[0]), array[1], array[2], token) <= 0;
}
```

و این هم لایهٔ اجرا که نتیجهٔ اعتبارسنجی را به کاربر نشان می‌دهد و تصمیم ادامه/توقف را برمی‌گرداند:

PersonCheckValidation.cs — نمایش نتیجهٔ اعتبارسنجی

```
public static async Task<DialogResult> CheckValidationAsync(
    string contextGuid, Person pObj, bool isShowMessage,
    CancellationToken token)
{
    try
    {
        ReturnedSaveFuncInfo val =
            await new PersonValidator(contextGuid, pObj).ValidateAsync(token);
        DialogResult result;
        if (val.HasError || val.HasInformation || val.HasWarning)
        {
            FrmShowErrorMessage val2 = new FrmShowErrorMessage(
                contextGuid, val, "", (MessageBoxButtons)3, false, ...);
            ((Form)val2).ShowDialog();      // نمایش خطاها به کاربر
            result = ((Form)val2).DialogResult;
        }
        else
        {
            result = (DialogResult)6;       // Yes → ادامهٔ ذخیره‌سازی
        }
        return result;
    }
    ...
}
```

در فرم پرسنل (فصل ۵) دیدیم که BtnAccept_Click اول همین متد را صدا می‌زند و فقط اگر نتیجه Yes بود به SaveAsync می‌رسد — یعنی ولیدیشن واقعاً اجباری است.

> **💡 شباهت به FluentValidation**
>
> این معماری با الگوی معروف **FluentValidation** در دنیای دات‌نت شباهت دارد: قواعد در کلاسی جدا از موجودیت و جدا از UI تعریف می‌شوند و خروجی یک «کیف نتیجه» (ReturnedSaveFuncInfo) است که هم خطا، هم هشدار و هم اطلاعات می‌تواند داشته باشد. مزیتش: همان قواعد در ذخیرهٔ دستی، ایمپورت اکسل و سینک ابری هم به کار می‌آیند.

فصل هشتم

CH-08

کتابخانهٔ Tele؛ مودم صوتی و تردد تلفنی

Tele.dll — AT Command Voice Modem

جذاب‌ترین کتابخانهٔ کوچک بسته! Tele (۲۳ کیلوبایت) یک **مودم صوتی** را از طریق پورت سریال کنترل می‌کند تا کارکنانی که دسترسی به دستگاه ندارند، با **تماس تلفنی** و واردکردن کد پرسنلی با کلیدهای گوشی، تردد ثبت کنند — کلاس اصلی آن به‌حق نام NovinPhone دارد. این کتابخانه دلیل حضور فایل System.IO.Ports.dll در بسته است.

زبان مودم‌ها **دستورات AT** است (AT از کلمهٔ Attention). مودم را با متن سریال_config می‌کنند و پاسخ OK یا ERROR می‌گیرند. جدول مهم‌ترین دستورات استفاده‌شده در این کتابخانه:

جدول ۸-۱: دستورات AT بکاررفته در Tele.dll

دستور AT	کارکرد در Tele	توضیح
`ATZ`	ریست مودم	بازگشت به تنظیمات کارخانه
`AT`	تست سلامت	پاسخ OK یعنی مودم زنده است
`AT+VCID=1`	فعال‌سازی شناسهٔ تماس‌گیرنده	Caller ID برای ثبت شمارهٔ تماس
`AT S0=3`	پاسخ خودکار بعد از ۳ زنگ	پیش‌فرض autoAnswerCount = 3
`AT+FCLASS=8`	ورود به حالت صوت	مودم به‌جای دیتا، صوت را مدیریت می‌کند
`ATD<شماره>;`	شماره‌گیری	تماس خروجی (مثلاً برای اطلاع‌رسانی)
`AT+VTX`	ارسال صوت خام	شروع استریم WAV به تماس‌گیرنده
`AT+VSM=9,8000`	تنظیم فرمت صوت	نمونه‌برداری ۸ کیلوهرتز

اجرای صوت فایل WAV برای شنیدن منوی صوتی، شاید آموزنده‌ترین بخش این کتابخانه باشد:

Tele/NovinPhone.cs — پخش فایل WAV روی خط تلفن

```
public void Play_Wave(string path)
{
    try
    {
        if (Status == ENStatus.PlayingVoice ||
            Status == ENStatus.Dissconnectd)
        {
            return;
        }
        StopPlaying = false;
        Status = ENStatus.PlayingVoice;
        VoiceMode();
        ExecCommnad("AT+FCLASS=8");      // حالت صوتی
        ExecCommnad("AT+VSM9,8000");     // فرمت ۸ کیلوهرتز
        ExecCommnad("AT+VSD\u00128,0");
        ExecCommnad("AT+VTX");           // شروع ارسال صوت
        byte[] array = File.ReadAllBytes(path);
        _serial.DtrEnable = true;
        int num = 44;                    // پرش از روی هدر ۴۴ بایتی WAV!
        int num2 = 1024;                 // تکه‌های ۱ کیلوبایتی
        while (true)
        {
            Thread.Sleep(80);            // ریتم ارسال: هر ۸۰ میلی‌ثانیه
            if (StopPlaying)
            {
                Status = ENStatus.PlayingVoiceFinished;
                StopPlaying = false;
                OnPlayingFinished();
                return;
            }
            _serial.Write(array, num, num2);
            num += num2;
            if (array.Length - num < 800)
            {
                num2 = array.Length - num;
            }
            if (num2 <= 0) break;
            Application.DoEvents();
        }
        ExecCommnad("AT+FCLASS=0");      // بازگشت به حالت دیتا
        OnPlayingFinished();
    }
```

```
    catch (Exception ex2) { ... }
}
```

نکتهٔ فنی: ۴۴ بایت اول فایل WAV هدرِ استاندارد RIFF است و مستقیم به مودم فرستاده نمی‌شود؛ فقط دادهٔ خام صوت ارسال می‌شود.

کلاس CLSNumberDetector گوش می‌دهد که تماس‌گیرنده چه اعدادی با کلیدهای گوشی فشرده است (دیتون‌های DTMF که به‌صورت کاراکتر از مودم می‌رسند) و هرچه تا فشردن * یا # تایپ شد را به‌عنوان کد پرسنلی برمی‌گرداند:

Tele/CLSNumberDetector.cs — دریافت کد پرسنلی از کلیدهای تلفن

```
public static string GetInput(NovinPhone phone, short maxSeconds)
{
    DateTime now = DateTime.Now;
    string text = "";
    while (Math.Abs(now.Subtract(DateTime.Now).Seconds) < maxSeconds)
    {
        Thread.Sleep(100);
        string text2 = phone.ReadLine();
        if (!(text2 == "") && !(text2 == "-"))
        {
            if (text2 == "*" || text2 == "#")
            {
                return text;          // پایان ورودی کاربر
            }
            text += ATCommandManager.GetInputNumber(text2);
        }
    }
    return text;                      // پایان مهلت زمانی
}
```

رویدادهای NovinPhone: Ring (زنگ)، Tone، EndCall، ConnectCall (هر کلید)، PlayingFinished و TerminalNewMessage. تمام وضعیت‌ها هم در enum وضعیت (ENStatus) مدیریت می‌شوند: زنگ‌خوردن، در حال تماس، پخش صوت، ضبط صوت...

🔎 ثبت کامل لاگ ارتباط سریال

متد GetInputNumber در ATCommandManager کار پاک‌سازی ورودی را انجام می‌دهد: پسوندهای H0/H1 و کاراکترهای + = کنار ارقام را حذف می‌کند و فقط رقم‌ها را نگه می‌دارد. همچنین کلاس FileManager هر گفتگوی سریال را در سه فایل Input.txt، Output.txt و ALL.txt کنار برنامه ثبت می‌کند — برای عیب‌یابی مودم بدون ابزار خاص.

فصل نهم

CH-09

کتابخانهٔ WebSmsStandard؛ ارسال پیامک

WebSmsStandard.dll — SMS REST Client

این کتابخانه (۲۶ کیلوبایت) کلاینت REST سرویس پیامکِ خود شرکت است و به آدرس http://webservice.novinp.ir/sms/Npsms/... متصل می‌شود. کارش ساده اما مهم است: در DAL دیدیم که موتور پیامک (ClsSms) برای غیبت‌های روزانه یا ترددهای ناقص، پیامک‌های خودکار می‌سازد؛ ارسال واقعیِ آن‌ها از همین‌جا انجام می‌شود.

مدل درخواست، ساده و JSON-محور است:

WebSmsStandard/SmsModelV2.cs — مدل درخواست ارسال

```
public class SmsModelV2 : SmsModelBase
{
    public List<string> Numbers { get; set; }    // شماره‌های گیرنده
    public List<string> Messages { get; set; }   // متن‌ها (نسبت ۱ به ۱)
}

// فیلدهای کلاس پایه SmsModelBase:
// UserName, Password, Domain, SenderNumber
```

WebSmsStandard/SmsSender.cs — ارسال با تلاش مجدد (۱۰ بار)

```
private static async Task<long[]> SendSmsAsync_(
    string Username, string Pass, string Domain, List<string> listMSG,
    List<string> listNum, string SenderNumber, CancellationToken inputToken,
    int tryCount = 10)
{
    ...
    SmsModelV2 smsModelV = new SmsModelV2
    {
        UserName = Username,
        Password = Pass,
        Domain = Domain,
        SenderNumber = SenderNumber,
        Messages = listMSG,
        Numbers = listNum
    };
    try
    {
        HttpResponseMessage response = await HttpHelpers.PostStreamAsync(
            (object)smsModelV,
            "http://webservice.novinp.ir/sms/Npsms/SendSmsV2/", token);
        ...
        if (response != null && response.IsSuccessStatusCode)
        {
            ActiveSendJsonResponse activeSendJsonResponse =
                await HttpHelpers.ReadJsonResponseAsync<ActiveSendJsonResponse>(response);
            if (activeSendJsonResponse != null &&
                activeSendJsonResponse.Result != null)
            {
                return activeSendJsonResponse.Result.ToArray();  // شناسه‌های پیامک
            }
        }
    }
    catch (OperationCanceledException)
    {
        if (tryCount > 0 && !inputToken.IsCancellationRequested)
        {
            await Task.Delay(new Random().Next(30000));   // مکث تصادفی تا ۳۰ ثانیه
            return await SendSmsAsync_(Username, Pass, Domain, listMSG,
                listNum, SenderNumber, inputToken, tryCount - 1);
        }
        return new long[1] { -119L };
    }
    ...
}
```

خروجی آرایه‌ای از long است: شناسهٔ هر پیامک برای پیگیری وضعیت تحویل؛ اعداد منفی هم کدهای خطای داخلی هستند (مثلاً -119 = لغو عملیات، -110 = خطای شبکه).

بخش آموزشیِ این کتابخانه، کلاس `RetryClientFactory` است که ترفند حرفه‌ای «تزریق هندلر retry در HttpClient» را نشان می‌دهد — الگویی که در پروژه‌های مدرن هم روز اول استفاده می‌شود:

WebSmsStandard/RetryClientFactory.cs — الگوی Retry در سطح HttpClient

```
internal class RetryClientFactory
{
    public class RetryHandler : DelegatingHandler
    {
        private readonly int _maxRetries = 3;
        private readonly int _delay = 2000;

        public RetryHandler(HttpMessageHandler innerHandler,
                            int maxRetries, int delay) : base(innerHandler)
        {
            _maxRetries = maxRetries;
            _delay = delay;
        }

        protected override async Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            HttpResponseMessage response = null;
            for (int i = 0; i < _maxRetries; i++)
            {
                response = await base.SendAsync(request, cancellationToken);
                if (response.IsSuccessStatusCode)
                {
                    return response;
                }
                // مکث نمایی با کمی تصادفی‌سازی (Jitter)
                await Task.Delay((int)((new Random().NextDouble() + 0.5) * _delay));
            }
            return response;
        }
    }

    public HttpClient GetClient()
    {
        return new HttpClient(new RetryHandler(new HttpClientHandler(),
            _maxRetries, _delay));
    }
}
```

💡 DelegatingHandler چیست؟

`DelegatingHandler` یک لایهٔ میانی در خط لولهٔ HttpClient است؛ مثل یک پستچی وسطی که هر درخواست را اول خودش بررسی می‌کند، به مقصد می‌فرستد و در صورت نیاز دوباره تلاش می‌کند. الگوی Retry با «مکث تصادفی» (Jitter) هم از هم‌زمان‌شدن هزاران تلاشِ مجدد پس از قطعی سرور جلوگیری می‌کند — همان چیزی که در کتابخانه‌های Polly و resilience مدرن هم هست.

متد دیگر، `GetDeliveryAsync` است که وضعیت تحویلِ پیامک‌های ارسال‌شده را با POST فرم-انکد (نه JSON) می‌پرسد و آرایه‌ای از کدهای وضعیت برمی‌گرداند؛ و `GetCreditAsync` موجودی پنل پیامک را نشان می‌دهد. همچنین کلاس OutBox_Other_Info مدل صف خروجی پیامک‌ها را در بانک محلی نگه می‌دارد.

CH-10 کتابخانه‌های ZK، ZDC2911 و zkemkeeper؛ دنیای بومی

ZK.dll / ZDC2911.dll / zkemkeeper.dll / zkemsdk.dll

۱۰-۱) ZK.dll؛ ابزار کوچک تست دستگاه

فایل ZK.dll (۱۰ کیلوبایت) یک برنامهٔ کوچک WinForms است (کلاس‌های Program و Form1 را دارد) که کارش خواندن ترددهای دستگاه ZKTeco است — عملاً نسخهٔ آزمایشی/جیبی‌شدهٔ همان ClsZK که در فصل ۴ دیدیم، به‌علاوهٔ یک کلاس Record با ۱۴ پراپرتی و پراپرتی محاسباتی DateM که اجزای سال/ماه/روز را به DateTime تبدیل می‌کند. جالب‌ترین بخشش این است که مستقیماً بعد از اتصال در سازنده، GetLog را اجرا می‌کند؛ یعنی «وصل شو و بلافاصله همه‌چیز را بخوان»!

ZK/ZK.cs — سازندهٔ کلاس ZK

```
public ZK(int MachineNum, string IP, int port, string Pass)
{
    _iMachineNumber = MachineNum;
    _IP = IP;
    _Port = port;
    _Pass = Pass;
    _bIsConnected = axCZKEM1.Connect_Net(IP, port);
    GetLog();     // خواندن فوری پس از اتصال!
}
```

پورت پیش‌فرض 5005 است — پورت استاندارد TCP دستگاه‌های ZKTeco.

۱۰-۲) ZDC2911.dll؛ SDK شرکت Realand

این DLL (۱۱۶ کیلوبایت) در واقع SDK نمونهٔ سازندهٔ Realand برای دستگاه‌های ZDC2911 است که تیم نوین پرداز آن را به‌صورت درون‌سازی (Embed) استفاده کرده. کلاس اصلی Realand یک فرم ویندوزی داخلی (CMForm) را بسته‌بندی می‌کند و از SDK مدیریت‌شدهٔ Riss.Devices استفاده می‌کند:

ZDC2911/Realand.cs — لایهٔ بسته‌بندی SDK

```
public class Realand
{
    private CMForm frm = null;

    public Realand(int DeviceNumber, string Password, string IP, int Port)
    {
        frm = new CMForm(DeviceNumber, Password, IP, Port);
    }

    public bool Connect()  { return frm.Connect(); }
    public bool Disconnect() { return frm.DisConnect(); }

    public List<Records> GetALLGlog()
    {
        List<Record> aLLGlog = frm.GetALLGlog();
        if (aLLGlog == null)
        {
            return null;
        }
        List<Records> list = new List<Records>();
        foreach (Record item in aLLGlog)
        {
            Records records = new Records();
            records.Action = item.Action;
```

```
            records.Antipassback = item.Antipassback;
            records.Clock = item.Clock;          // زمان تردد
            records.DIN = item.DIN;              // شمارهٔ کاربر
            records.DN = item.DN;                // شمارهٔ دستگاه
            records.Verify = item.Verify;        // نوع تأیید
            list.Add(records);
        }
        return list;
    }
}
```

نام‌گذاری فیلدهای Record در این SDK (DN/DIN/Clock/Verify/MDIN) دقیقاً همان SDK Riss است؛ این دو خانوادهٔ سخت‌افزاری خویشاوندند! ZDC2911 همچنین ۱۰ فرم UI نمونه (RealTimeLogForm، EnrollForm، GLogForm...) دارد که نشان می‌دهد SDK به‌صورت یک برنامهٔ دمو هم قابل اجراست.

۱۰-۳) zkemkeeper.dll و zkemsdk.dll؛ قلب بومی ZKTeco

این دو فایل با هم ۸۵۸ کیلوبایت و برخلاف بقیهٔ DLL های بسته، **دات‌نت نیستند** — فایل‌های بومی ۳۲ PEبیتی (x86) هستند. `zkemkeeper.dll` همان SDK رسمی ZKTeco است که کلاس COM معروف **CZKEM** را ارائه می‌دهد؛ یعنی همان چیزی که ClsZK و ZK.dll از طریق Interop با آن کار می‌کنند. `zkemsdk.dll` هم لایهٔ پایین‌تر همان خانواده است. با اسکن نمادهای داخلی این دو فایل، فهرست توابع زیر شناسایی شد:

جدول ۱۰-۱: توابع نمونهٔ داخل zkemkeeper.dll و zkemsdk.dll (استخراج با اسکن نمادها)

نمونهٔ تابع بومی	کارکرد
`Z_Connect_NET / Z_Connect_COM / Z_Connect_USB`	اتصال از راه شبکه / پورت سریال / USB
`Z_ReadLogEx / Z_GetLog / Z_GetOpLog`	خواندن ترددها و لاگ عملیات
`Z_GetUserInfo / Z_SetUserInfo`	خواندن/نوشتن اطلاعات کاربران
`Z_GetUserTmp / Z_SetUserTmp`	انتقال قالب اثر انگشت
`Z_EnableDevice / Z_DisableDevice`	قفل/آزادسازی دستگاه هنگام انتقال داده
`Z_GetDeviceStatus / Z_GetDeviceInfo`	وضعیت و مشخصات دستگاه
`Z_SetTime / Z_GetTime`	همگام‌سازی ساعت دستگاه
`Z_PowerOffDevice / Z_RebootDevice`	خاموش/ریست دستگاه از راه دور
`Z_ClearLogData`	پاک‌کردن تاریخچهٔ ترددها
`_SSR_Z_GetUserInfo ...`	توابع سری SSR (SSR_GetGeneralLogData و...)

💡 نصب و استفاده از COM بومی

DLL های بومی ۳۲بیتی مثل این‌ها باید قبل از استفاده در ویندوز «ثبت» شوند: دستور `regsvr32 zkemkeeper.dll` آن‌ها را در رجیستری معرفی می‌کند تا بقیهٔ برنامه‌ها بتوانند از COM آن‌ها شیء بسازند. دو نکتهٔ عملی: (۱) چون ۳۲بیتی‌اند، برنامهٔ میزبان هم باید با پلتفرم x86 کامپایل شده باشد وگرنه خطای می‌گیرد؛ (۲) تابع نهایی‌شان معمولاً اسم مدل دستگاه را لاگ می‌کند — راه‌انداز رسمی همراه با نصب‌کنندهٔ SDK خود ZKTeco روی سیستم باید کپی شود. برای کار مستقیم با این SDK در پروژهٔ خودتان، بعد از افزودن ارجاع COM به zkemkeeper، ویژوال استودیو Interop آن را می‌سازد.

CH-11

کتابخانه‌های جانبی؛ اعلان و انیمیشن

Tulpep.NotificationWindow + WinFormAnimation

دو کتابخانهٔ متن‌باز این بسته، از مخزن عمومی NuGet آمده‌اند و کارشان «زیباسازی» تجربهٔ کاربری است — هرچند نقش‌شان در این سیستم پررنگ است، چون **همهٔ اعلان‌های نرم‌افزار** (از «تردد جدید دریافت شد» گرفته تا خطاهای ذخیره‌سازی) با این دو نمایش داده می‌شوند.

۱۱-۱) Tulpep.NotificationWindow؛ اعلان شیشه‌ای ویندوز

این کتابخانه (۲۸ کیلوبایت) کامپوننتی به نام PopupNotifier ارائه می‌دهد: پنجرهٔ اعلانی گوشهٔ پایین صفحه با عنوان، متن، آیکن و دکمهٔ بستن — همان چیزی که همهٔ ما به‌عنوان «نوتیفیکیشن ویندوز» می‌شناسیم. ساختارش آموزشی است: یک Component با ده‌ها پراپرتی قابل تنظیم (رنگ سربرگ، رنگ متن، فونت، مدت نمایش، سرعت انیمیشن...) و یک فرم داخلی (PopupNotifierForm) که خودش را بدون فعال‌شدنِ پنجرهٔ اصلی نشان می‌دهد — با تابع Win32 به نام ShowWindow و پرچم SW_SHOWNOACTIVATE = 4. تایمری هم (tmrAnimation) موقعیت و شفافیت پنجره را قدم‌به‌قدم تغییر می‌دهد تا اعلان «سُر بخورد» بالا.

Tulpep.NotificationWindow/PopupNotifier.cs — اعلان شیشه‌ای با تایمر انیمیشن

```
[ToolboxBitmap(typeof(PopupNotifier), "Icon.ico")]
[DefaultEvent("Click")]
public class PopupNotifier : Component
{
    private const int SW_SHOWNOACTIVATE = 4;
    private const int HWND_TOPMOST = -1;
    private const uint SWP_NOACTIVATE = 16u;

    private PopupNotifierForm frmPopup;
    private Timer tmrAnimation;
    private Timer tmrWait;
    private bool isAppearing = true;

    [Description("Color of the window header.")]
    [Category("Header")]
    [DefaultValue(typeof(Color), "ControlDark")]
    public Color HeaderColor { get; set; }
    ...
}
```

استفادهٔ ویژگی‌های Description/Category/DefaultValue یعنی این کامپوننت برای «جعبه‌ابزار ویژوال استودیو» طراحی شده — در Designer می‌افتد و با موس تنظیم می‌شود.

۱۱-۲) WinFormAnimation؛ انیمیشن سبک‌وزن فرم‌ها

کتابخانهٔ WinFormAnimation (۳۸ کیلوبایت) موتور حرکت است: کلاس‌های Animator (یک‌بعدی)، Animator2D (دوبعدی) و Animator3D به‌همراه Path ها که مسیر حرکت (مثلاً از نقطهٔ الف تا ب با شتاب اولیه) را توصیف می‌کنند. در این سیستم، جفتِ این دو کتابخانه اعلان‌ها را «سُر می‌دهد»: PopupNotifier پنجره را می‌سازد و انیمیشنِ بالا آمدن/محو شدن را با همین کتابخانه انجام می‌دهد. تابع‌های انیمیشن آمادهٔ آن (KnownAnimationFunctions) شامل شتاب‌های خطی، درجه‌دو، Sine و EaseOut است.

در کد دکامپایل‌شده، نقطهٔ اتصال این دو کتابخانه به سیستم اصلی، کلاس‌های ClsNotif و MyPopupForm در فضای‌نام TimeAttendanceDAL.Utility هستند[۳۶] که همهٔ فرم‌ها از طریق Utility.MessageBoxShowByNotif(...) با آن

صدا می‌زنند — یعنی UI حتی برای نمایش یک پیام ساده هم به DAL وابسته است! (نکتهٔ معماری: بهتر بود این لایهٔ کمکی جدا بود.)

> **💡 چرا اعلان شیشه‌ای به‌جای MessageBox؟**
>
> چرا از MessageBox معمولی ویندوز استفاده نکردند؟ چون MessageBox «بلاک‌کننده» است: تا دکمهٔ OK را نزنید نمی‌توانید با برنامه کار کنید. در سیستم حضور و غیاب که هر چند ثانیه ممکن است تردد جدید برسد، انبوه پیام‌های بلاک‌کننده یعنی قفل‌شدن کار کاربر. اعلان‌های شیشه‌ای خودمحو (بعد از چند ثانیه محو می‌شوند) راه‌حل درستِ این مسئله‌اند.

فصل دوازدهم

CH-12

کتابخانه‌های System.* مایکروسافت؛ زیرساخت

Microsoft BCL Libraries (26 files)

بیست‌وشش فایل با پیشوند System. در این بسته، هیچ‌کدام اختصاصی نیستند؛ این‌ها کتابخانه‌های استاندارد مایکروسافت‌اند که توسعه‌دهنده کنار برنامه کپی کرده تا روی سیستم‌های مقصد نصب جداگانه لازم نباشد (روش معروف XCOPY Deploy). در جدول زیر مهم‌ترین‌شان را با نقششان در «این سیستم خاص» مرور می‌کنیم — چون دیدن این مجموعه خودش درس آموزشی خوبی است: می‌بینید یک نرم‌افزار ویندوزی واقعی برای کار کردن به کدام قطعات استاندارد نیاز دارد.

جدول ۱۲-۱: مرور کتابخانه‌های مایکروسافتی بسته (خلاصه‌شده بر اساس نقش)

کتابخانه	نقش عمومی	چرا این سیستم به آن نیاز دارد
`System.Data.SqlClient`	کلاینت SQL Server (ADO.NET)	همهٔ کارهای بانک اطلاعاتی DAL
`Encoding.CodePages` + `Encodings.Web` + `System.Text.Json`	سریالایز JSON و صفحه‌کدها	تنظیمات JSON، پروتکل Hysoon، خروجی ابری؛ CodePages برای صفحه‌کد فارسی 1256
`Numaics.Vectors` + `System.Memory` + `ValueTuple` + `Runtime.CompilerServices.Unsafe` + `Tasks.Extensions`	پیش‌نیازهای عملکردی جدید در دات‌نت قدیمی	Span، تاپل، async در NET Framework 4.8
`System.IO.Ports`	پورت سریال (سریال‌پورت)	کتابخانهٔ Tele برای مودم صوتی
`System.Drawing.Common`	گرافیک GDI+	رندر فرم‌ها، تصاویر پرسنل، اسکرین‌شات
`System.IdentityModel.Tokens.Jwt`	توکن‌های JWT	احراز هویت با سرور ابری نوین پرداز
`System.Collections.Immutable`	کالکشن‌های تغییرناپذیر	لیست‌های امن در چندتردی
`Tasks.Dataflow` + `System.Threading.Channels`	صف‌های تولیدکننده/مصرف‌کننده	صف اعلان‌ها و BufferBlock ترددهای زنده
`Net.WebSockets.WebSocketProtocol` + `System.IO.Pipelines`	I/O و وب‌سوکت مدرن	زیرساخت SignalR (ارتباط ابری)

کتابخانه	نقش عمومی	چرا این سیستم به آن نیاز دارد
`System.Configuration.ConfigurationManager`	خواندن فایل‌های config	همان فایل‌های .config کنار DLL ها
`System.Security.*` (۴ فایل)	کریپتوگرافی و رمزنگاری AES در ACL ویندوز	DAL و دسترسی فایل
`ComponentModel.Annotations` + `System.CodeDom` + `Diagnostics.DiagnosticSource`	تولید کد، دیتا-انوتیشن، ردیابی	فرمول‌های حقوق، اعتبارسنجی، لاگ ردیابی

> 🔎 **فسیل‌شناسی نسخه‌ها**
>
> نسخهٔ فایل‌ها از 2018 تا 2024 متغیر است (مثلاً System.Text.Json از سپتامبر 2024 ولی System.Memory از 2022) — نشانهٔ این که طی سال‌ها با آپدیت بسته‌های NuGet، DLL های تازه‌تر جایگزین شده‌اند بدون اینکه کل پروژه بازنویسی شود. این «زندگی طولانی» پروژه‌های WinForms صنعتی است: از 2013 (سال کپی‌رایت در کدها) تا امروز!

فصل سیزدهم

CH-13

جمع‌بندی؛ درس‌ها و یافته‌های امنیتی

Lessons & Security Findings

۱۳-۱) آنچه یاد گرفتیم

در این سفر، یک نرم‌افزار صنعتی کامل را کالبدشکافی کردیم: دیدیم چگونه یک معماری چندلایهٔ WinForms + SQL Server با ده‌ها فرم، شش برند سخت‌افزار، سینک ابری SignalR، پیامک خودکار و حتی مودم تلفنی، در قالب ۱۳ کتابخانهٔ اختصاصی سازمان‌دهی شده است. الگوهای مهمی که دیدیم می‌تواند چراغ راه پروژه‌های شما باشد:

- **تفکیک مسئولیت:** UI هیچ SQL ای اجرا نمی‌کند؛ همه‌چیز از DAL با روال‌های ذخیره‌شده می‌گذرد — دقیقاً همان‌طور که کتاب‌های معماری توصیه می‌کنند.
- **قرارداد دادهٔ مشترک:** با کلاس DeviceLogOnPerson، هفت مسیر سخت‌افزاری متفاوت به یک شکل در می‌آیند.
- **تحمل خطا:** try/catch سراسری + لاگ WebErrorLog + تلاش مجدد بازگشتی؛ نرم‌افزار صنعتی باید «نمرده» بماند.
- **کش هوشمند:** کش ۶۰ ثانیه‌ای پرسنل و Singleton بر اساس contextGuid.
- **رمزنگاری با کلید مناسب:** پیاده‌سازی AES درست بود اما کلیدش داخل کد بود — قفل فولادی روی در پارچه‌ای.
- **خودکارسازی نصب:** از ساخت بانک و اجرای اسکریپت‌های شِما تا پرکردن تقویم شمسی و جدول مالیات، همه خودکار.

۱۳-۲) کارنامهٔ امنیتی

جدول ۱۳-۱: یافته‌های امنیتی گزارش‌شده از کد دکامپایل‌شده

یافته	محل	شدت	توضیح
رمزهای استاد ورود	FrmEnter.cs (UI)	بحرانی	دو رمز ثابت که با هر کاربری ورود می‌دهند
کلید رمزنگاری هاردکد	UserPermission.cs (DAL)	بالا	عبارت "NovinP**********" برای AES
تزریق SQL	Shifts.cs و چند متد دیگر	متوسط	چسباندن رشته به کوئری به‌جای پارامتر
توکن ربات تلگرام	Utility.cs (DAL)	متوسط	توکن کامل + ارسال خودکار مشخصات نصب
کاربر مدیر بدون رمز	UserAttendance.cs	متوسط	در اولین اجرا کاربری «مدیر» با رمز خالی ساخته می‌شود

یافته	محل	شدت	توضیح
فرم دسترسی ویژه بدون رمز	FrmManagment_Login.cs	بالا	اجرای مستقیم روال‌های SQL بدون احراز هویت
اتصال SMS روی HTTP	SmsSender.cs	کم	ارسال نام کاربری/رمز بدون رمزگذاری انتقالی (http نه https)

این یافته‌ها را «برای سوءاستفاده» نبینید؛ برای **یادگیری** هستند. اگر روزی خودتان سیستمی با این مقیاس ساختید، همین هفت مورد چک‌لیست بازبینی امنیتی شماست: رمز استاد نگذارید، کلید را در ماژول امن نگه دارید، همیشه کوئری پارامتری بنویسید، توکن‌ها را بیرون از کد نگه دارید، کاربر پیش‌فرض با رمز قوی یا بدون رمز نسازید، فرم‌های مدیریتی را احراز هویت کنید و ترافیک حساس را روی TLS ببرید.

۱۳-۳) اگر بخواهیم ادامه دهیم...

- می‌توانید فایل‌های .pdb موجود در بسته را در ILSpy یا dnSpy کنار DLL ها باز کنید تا شماره خطوط سورس اصلی را هم ببینید.
- قسمت‌های محاسباتی TaradodFullDet (۴٬۰۰۰+ خط) ارزش خواندن تک‌تک را دارد: جفت‌کردن ورود/خروج، شیفت شب، جمعه‌کاری، مرخصی ساعتی...
- برای آزمایش امنیتی خودتان: از بستهٔ SQL Server و بانک AttendanceN یک نمونهٔ آزمایشی بسازید و اسکریپت‌های Create_* را اجرا کنید تا شِما را از نزدیک ببینید.
- اگر پروژه‌ای مشابه دارید، مطالعهٔ الگوی «کلاس‌های کمکی کوچک» (مثل SmsCalendar) برای شروع عالی است؛ سپس سراغ کلاس‌های غول‌پیکر (Fish با ۸۵KB کد) بروید تا ببینید چطور «نباید» بزرگ شد!

۱۳-۴) چک‌لیست معلم؛ برای پروژهٔ بعدی شما

اگر خلاصهٔ این گزارش را در ده بند بخواهیم — چک‌لیستی که می‌توانید هنگام طراحی یا بازبینی هر نرم‌افزار سازمانی کنار دستتان بگذارید:

- لایه‌ها را جدا نگه دارید: UI فقط نمایش، DAL فقط داده — و بین‌شان قرارداد مشخص (متدهای async با contextGuid).
- برای هر جدول، چهار روال استاندارد بسازید (Insert/Update/SelectRow/SelectAll) و قاعدهٔ نام‌گذاری را هرگز نشکنید.
- روی داده‌های پرخواننده (مثل فهرست پرسنل) کش با انقضای زمانی بگذارید و آن را زودتر از موعد باطل نکنید.
- هر عملیات حساس (ذخیره، حذف، تغییر دسترسی) را با لاگ حسابرسی همراه کنید: چه‌کسی، چه‌زمانی، چه چیزی.
- کلید و رمز را از کد جدا کنید؛ هاردکدکردن passphrase یعنی دادن کلید به هر خوانندهٔ دیکامپایلر.
- کوئری همیشه پارامتری — حتی برای یک عدد ساده؛ تزریق SQL از همین «موارد استثنای کوچک» وارد می‌شود.
- برای ارتباط زنده با سخت‌افزار، از رویداد/Push استفاده کنید و در نبود آن، پولینگ با فاصلهٔ معقول بگذارید.
- پیام‌های کاربر را در لایهٔ مناسب و به زبان کاربر تولید کنید؛ فارسی‌نویسی تمیز نیمی از تجربهٔ کاربری است.
- خطا را ببلعید اما گم نکنید: try/catch سراسری + لاگ ساخت‌یافته (Serilog) + تلاش مجدد با مکث تصادفی.
- پیش از انتشار، یک بار خودتان را جای مهاجم بگذارید و همان هفت سطر جدول ۱۳-۱ را نقض ناپذیر کنید.

امیدواریم این گزارش، چراغی روشن در مسیر یادگیری معماری نرم‌افزارهای واقعی بوده باشد. کد واقعی همیشه پر از درس‌های نهان است — چه در نکات درستش و چه در اشتباه‌هایش. موفق باشید!

پایان سفر،
آغاز تمرین

حالا که معماری یک نرم‌افزار صنعتی واقعی را از درون دیده‌اید، بهترین قدم بعدی، باز کردن ILSpy و کاوش خودتان است. کد واقعی، بهترین معلم است.

۱۳	+۴۵۰	+۶۰	۷
کتابخانهٔ تحلیل‌شده	کلاس بررسی‌شده	قطعه کد واقعی	یافتهٔ امنیتی

گزارش آموزشی تحلیل کتابخانه‌های سیستم حضور و غیاب نوین پرداز — آبان ۱۴۰۵